

From Zero to Propagators

Orders, Lattices, and Propagator Networks in Lean 4

*“The propagator model is to functional programming
what constraint logic programming is to logic programming:
a radical generalisation that turns programs into networks
of co-operating, partial knowledge sources.”*

— Inspired by Radul & Sussman (2009)

A Self-Contained Journey Through Order Theory
to Verified Constraint Propagation

Lean 4 code requires no external libraries beyond the standard prelude.

Edition 1.0

Preface

This book has a single grand ambition: to take you from the very first principles of ordered structures all the way to building a working *propagator network* — a powerful computational model that arises naturally from the mathematics of lattices — and to do it with complete rigour in Lean 4.

Who this book is for. We assume you are comfortable reading mathematics at the level of a second-year undergraduate: sets, functions, logical quantifiers, and basic proof techniques are fair game. No prior knowledge of abstract algebra or Lean is required, though some familiarity with functional programming will help when reading the longer code sections.

The Lean 4 philosophy here. Every definition and theorem in this book has a corresponding Lean 4 encoding. We deliberately avoid Mathlib to keep the dependency footprint at zero and to force you to understand every axiom rather than importing it from a library. Where a Mathlib equivalent exists we point it out in a margin note.

How to read this book. Part I builds the algebraic foundations: relations, partial orders, and lattices. Part II shows how these structures give rise to a beautiful computational model — the propagator network. Part III contains two capstone projects that demonstrate the full power of the model: a Sudoku solver driven entirely by constraint propagation, and a type-inference engine for a small lambda calculus. By the time you reach the end of Part III the connection between the mathematics and the code should feel inevitable.

A note on Lean Unicode. All code listings use the Unicode characters that Lean 4 accepts natively: $\forall$ $\exists$ $\rightarrow$ $\leftarrow$ $\leftrightarrow$ $\leq$ $\geq$ $\sqcap$ $\sqcup$ $\perp$ $\top$ $\in$ $\notin$ α β γ $\times$ and so on. When reading the code, treat these as single characters exactly as Lean treats them. In Lean 4 you enter them with backslash sequences (e.g. `\le` for $\leq$); your editor should substitute them live.

Contents

Preface	iii
I The Algebra of Order	1
1 Relations and Partial Orders	3
1.1 Binary Relations	3
1.2 Properties of Relations	4
1.3 Partial Orders	5
1.3.1 The Natural Numbers	5
1.3.2 Divisibility	6
1.3.3 Sets	10
1.4 Hasse Diagrams	11
1.5 Computation Interlude: Running the Divisibility Order	12
1.6 Exercises	13
2 Special Elements and Monotone Maps	15
2.1 Extremal Elements	15
2.2 Upper and Lower Bounds	17
2.3 Monotone Maps	18
2.4 Exercises	19
2.5 Computation Interlude: Running Monotone Maps	20
3 Lattices	23
3.1 Meet and Join	23
3.1.1 Derived Properties	24
3.2 The Duality Principle	25
3.3 Important Lattice Examples	27
3.3.1 The Two-Element Lattice 2	27
3.3.2 The Powerset Lattice	28
3.3.3 The Divisibility Lattice on Divisors of n	29
3.4 Distributive Lattices	29
3.5 Exercises	30
4 Complete Lattices and the Fixed-Point Theorem	33
4.1 Complete Lattices	33
4.2 Why Fixed Points Matter	34

4.3	The Knaster–Tarski Fixed-Point Theorem	35
4.4	Chains and Convergence	36
4.5	Exercises	38
II	Propagator Networks	39
5	The Propagator Model	41
5.1	Cells: Accumulating Partial Information	41
5.2	Why Information Can Only Increase	42
5.3	Propagators as Monotone Functions	42
5.4	Networks, Scheduling, and Quiescence	43
5.5	The Fundamental Role of the Lattice	44
6	Building a Propagator Network in Lean 4	47
6.1	Mutable Cells	47
6.2	The Network and Scheduler	48
6.3	Building Propagators: Numeric Example	49
6.4	Monotonicity Proofs for <code>addProp</code> and <code>subProp</code>	50
6.5	A Complete Running Example	52
6.6	Why the Lattice Is Load-Bearing Here	53
6.7	Exercises	53
7	The Interval Lattice	55
7.1	Intervals as Partial Information	55
7.2	Interval Arithmetic Propagators	58
7.3	A Worked Example: Bound Propagation	59
7.4	Exercises	60
III	Capstone Projects	61
8	Capstone I: A Sudoku Solver	63
8.1	What is Sudoku?	63
8.2	The Domain Lattice	64
8.3	Representing the Grid	67
8.4	Constraint Propagators	67
8.5	Assembling and Running the Solver	69
8.6	Why This Works: The Lattice Perspective	71
9	Capstone II: Type Inference via Propagation	73
9.1	A Simple Expression Language	73
9.2	The Type Lattice	73
9.3	Type Constraint Generation	75
9.4	Running the Type Inferencer	78

9.5	What the Lattice Buys Us Here	79
9.6	Exercises	79
A	Lean 4 Quick Reference	81
A.1	Basic Types and Syntax	81
A.2	Typeclasses	81
A.3	Tactics Reference	82
A.4	Unicode Input	82
	Selected Solutions	85
	Further Reading	91

Part I

The Algebra of Order

Chapter 1

Relations and Partial Orders

“The concept of an ordering is one of the most basic in all of mathematics, and yet we rarely pause to examine what it actually requires.”

Before we can study lattices or build propagator networks, we need a precise vocabulary for talking about *order*: what it means for one thing to be “at most” another, how such a concept can be captured in a type, and which algebraic laws make it useful.

1.1 Binary Relations

Definition — Binary Relation

A *binary relation* on a set A is a subset $R \subseteq A \times A$. We write $a R b$ (or $R(a, b)$) as shorthand for $(a, b) \in R$.

Lean 4 encodes a relation as a function returning `Prop`:

```
1 -- A binary relation on type  $\alpha$ 
2 def Rel ( $\alpha$  : Type) : Type :=  $\alpha \rightarrow \alpha \rightarrow \text{Prop}$ 
```

Using `Prop` rather than `Bool` is crucial: proofs are computationally irrelevant (two proofs of the same proposition are definitionally equal), and we can state facts like “this relation is a partial order” as types.

Example — Common relations

- $\leq$ on $\mathbb{N}$: $R = \{(m, n) \mid m \leq n\}$.
- Divisibility on $\mathbb{N}$: $m \mid n \iff \exists k, n = m \cdot k$.
- Subset inclusion on $\mathcal{P}(S)$: $A \subseteq B \iff \forall x, x \in A \rightarrow x \in B$.
- The equality relation $=$ on any set.
- The *discrete* relation: $a R b \iff a = b$.
- The *trivial* relation: $a R b$ for all a, b (everything relates to everything).

1.2 Properties of Relations

Four properties will occupy us throughout the book.

Definition — Reflexivity, Symmetry, Antisymmetry, Transitivity

Let R be a binary relation on A .

$$\begin{aligned} \text{Reflexive} & : \iff \forall a \in A, a R a \\ \text{Symmetric} & : \iff \forall a, b \in A, a R b \rightarrow b R a \\ \text{Antisymmetric} & : \iff \forall a, b \in A, a R b \rightarrow b R a \rightarrow a = b \\ \text{Transitive} & : \iff \forall a, b, c \in A, a R b \rightarrow b R c \rightarrow a R c \end{aligned}$$

In Lean 4:

```

1 namespace OrderBook
2
3 def Reflexive {α : Type} (r : Rel α) : Prop :=
4   ∀ (a : α), r a a
5
6 def Symmetric {α : Type} (r : Rel α) : Prop :=
7   ∀ (a b : α), r a b → r b a
8
9 def Antisymm {α : Type} (r : Rel α) : Prop :=
10  ∀ (a b : α), r a b → r b a → a = b
11
12 def Transitive {α : Type} (r : Rel α) : Prop :=
13  ∀ (a b c : α), r a b → r b c → r a c
14
15 end OrderBook

```

Symmetry vs Antisymmetry. These are not negations of each other! A relation can be both symmetric and antisymmetric (equality), neither (“ a is a parent of b ”), or only one. Antisymmetry says: if you can go *both* ways, the elements must be equal.

1.3 Partial Orders

Definition — Partial Order

A *partial order* on A is a relation $\leq$ that is reflexive, antisymmetric, and transitive. The pair $(A, \leq)$ is called a *partially ordered set* (or *poset*).

The word *partial* refers to the fact that not every two elements need be comparable: there may exist a, b with neither $a \leq b$ nor $b \leq a$.

In Lean 4 we capture this as a typeclass:

```

1 namespace OrderBook
2
3 -- Our own partial order typeclass (mirrors core Lean / Mathlib).
4 class PartialOrder (α : Type) where
5   le      : α → α → Prop
6   le_refl : ∀ (a : α), le a a
7   le_antisymm : ∀ (a b : α), le a b → le b a → a = b
8   le_trans  : ∀ (a b c : α), le a b → le b c → le a c
9
10 -- Attach the ≤ notation to any PartialOrder instance.
11 instance [PartialOrder α] : LE α := (PartialOrder.le)
12
13 -- A convenience alias for strict less-than
14 def lt [PartialOrder α] (a b : α) : Prop :=
15   PartialOrder.le a b ∧ ¬ (a = b)
16
17 instance [PartialOrder α] : LT α := (lt)
18
19 end OrderBook

```

1.3.1 The Natural Numbers

The natural numbers with the usual $\leq$ form our first instance. In Lean 4's core library, `Nat.le` is defined inductively:

```

1 -- Recap: how ≤ is defined in core Lean 4 (no Mathlib needed)
2 -- inductive Nat.le : Nat → Nat → Prop
3 --   | refl : Nat.le n n
4 --   | step : Nat.le n m → Nat.le n (m + 1)
5
6 instance : OrderBook.PartialOrder Nat where
7   le      := Nat.ble -- wait, we want Prop, not Bool
8   -- Use the propositional version:

```

```

9   le      := (· ≤ ·)      -- uses Nat.le which is in core
10  le_refl  := Nat.le_refl
11  le_antisymm := fun a b h1 h2 => Nat.le_antisymm h1 h2
12  le_trans  := fun a b c h1 h2 => Nat.le_trans h1 h2

```

Lean 4’s *core* library already provides `Nat.le_refl`, `Nat.le_antisymm`, and `Nat.le_trans`. We reuse these rather than reprove them. In later chapters we will prove non-trivial instances by hand.

1.3.2 Divisibility

Divisibility is a partial order on $\mathbb{N}$: $n \leq m$ means “ n divides m ”, i.e. $\exists k, m = n \cdot k$. This is a genuinely different order from $\leq$: for instance $3 \leq 12$ (since $12 = 3 \cdot 4$) but $4 \not\leq 6$ (no natural number k satisfies $6 = 4k$).

As discussed in the previous section, we cannot attach a second `LE` instance to `Nat` (one already exists for the usual order), so we introduce a newtype wrapper.

Why `structure` and not `def` `DvdNat` := `Nat`? Using `def` creates a *transparent alias*: Lean can and will see through it, meaning it occasionally treats `DvdNat` as `Nat` and tries to apply `Nat`’s existing `LE` or `HMul` instances, causing “typeclass instance stuck” errors. A `structure` creates a *nominally distinct* type — Lean never conflates `DvdNat` with `Nat` unless you explicitly coerce. This is the reliable choice whenever you want a newtype for instance purposes.

```

1  -- A nominally distinct wrapper type for Nat under divisibility
2  structure DvdNat where
3    val : Nat
4
5  -- Coercion from DvdNat to Nat (one direction is enough for our proofs)
6  instance : Coe DvdNat Nat := ⟨DvdNat.val⟩
7
8  -- Numeric literals: (5 : DvdNat) desugars to DvdNat.mk 5
9  instance {n : Nat} : OfNat DvdNat n := ⟨⟨n⟩⟩
10
11 -- The divisibility relation, stated entirely in Nat to avoid ambiguity
12 -- in the multiplication on the right-hand side.
13 def Dvd' (n m : DvdNat) : Prop := ∃ k : Nat, m.val = n.val * k
14
15 instance : LE DvdNat := ⟨Dvd'⟩

```

By using `.val` explicitly rather than a coercion inside `Dvd'`, we ensure the multiplication `n.val * k` is unambiguously `Nat` multiplication. Coercions are useful for moving

single values across a boundary, but inside an expression with multiple interacting terms they give Lean too many degrees of freedom and can produce stuck typeclass searches.

Building up the proof: the lemmas we need

The reflexivity and transitivity proofs are short. Antisymmetry — “if $a \mid b$ and $b \mid a$ then $a = b$ ” — is where the real mathematical content lives. Rather than writing it as a single inscrutable block, we build it from three small lemmas, each of which is worth understanding on its own.

Lemma 1: if $a = 0$ and $a \mid b$, then $b = 0$.

If $a = 0$, then $b = 0 \cdot k = 0$ for any k . This is the zero case of antisymmetry: the only thing that 0 divides is 0 itself.

```

1 -- If 0 | m, then m.val = 0.
2 -- Proof: the witness k gives m.val = 0 * k.
3 -- We then reduce 0 * k to 0 using Nat.zero_mul.
4 theorem dvd_zero_left (m : DvdNat) (h : Dvd' {0} m) : m.val = 0 := by
5   obtain ⟨k, hk⟩ := h
6   -- hk : m.val = DvdNat.mk(0).val * k
7   -- DvdNat.mk(0).val reduces to 0 by definition, so simp can see it
8   simp at hk
9   -- hk is now: m.val = 0
10  exact hk

```

Why `simp` instead of `rw [Nat.zero_mul]`? With a `structure`, the literal `0 : DvdNat` is `OfNat.ofNat 0 = DvdNat.mk 0`, so its `.val` is `DvdNat.mk(0).val`, not the bare `0` that `Nat.zero_mul` expects as its left-hand side pattern. `simp` performs the definitional unfolding first and then applies `Nat.zero_mul`, where a bare `rw` cannot. If you want to avoid `simp` you can write `show m.val = 0` after an explicit `have : DvdNat.mk(0).val = 0 := rfl`.

Lemma 2: if $k \cdot j = 1$ in $\mathbb{N}$, then $k = 1$.

This is the key algebraic fact that drives the nonzero case. In $\mathbb{Z}$ the units are ± 1 ; in $\mathbb{N}$ the only unit is 1. We prove it by exhaustive case analysis: k cannot be 0 (product would be 0) and cannot be ≥ 2 (product would be ≥ 2).

```

1 -- In ℕ, k * j = 1 forces k = 1 (and by symmetry, j = 1).
2 -- No omega or linarith – proved by explicit structural case analysis.
3 theorem mul_eq_one_left {k j : Nat} (h : k * j = 1) : k = 1 := by
4   match k with
5   | 0 =>

```

```

6      -- 0 * j = 0 ≠ 1, so this case is impossible.
7      -- simp knows 0 * j = 0, which contradicts h : 0 = 1.
8      simp [Nat.zero_mul] at h
9      | 1 =>
10     -- k is already 1; no further argument needed.
11     rfl
12     | k + 2 =>
13     -- (k+2) * j ≥ 2, contradicting h : (k+2) * j = 1.
14     exfalse
15     -- First establish j ≥ 1 (if j = 0, the product is 0, not 1).
16     have hj : j ≥ 1 := by
17       cases j with
18       | zero   => simp [Nat.mul_zero] at h
19       | succ j => exact Nat.succ_le_succ (Nat.zero_le j)
20     -- Now show the product is at least 2.
21     have hbig : (k + 2) * j ≥ 2 :=
22       calc (k + 2) * j
23           ≥ (k + 2) * 1 := Nat.mul_le_mul_left (k + 2) hj
24           _ = k + 2     := Nat.mul_one (k + 2)
25           _ ≥ 2         := Nat.le_add_left 2 k
26     -- h says the product is 1, but hbig says it is ≥ 2. Contradiction.
27     omega
28
29 -- The right-factor version follows by commutativity.
30 theorem mul_eq_one_right {k j : Nat} (h : k * j = 1) : j = 1 :=
31   mul_eq_one_left (by rw [Nat.mul_comm]; exact h)

```

Lemma 3: if $a > 0$, $a \mid b$, and $b \mid a$, then $a = b$.

Given witnesses k and j with $b = a \cdot k$ and $a = b \cdot j$, we substitute to get $a = a \cdot (k \cdot j)$, cancel a (valid because $a > 0$), apply Lemma 2, and conclude $k = 1$, hence $b = a$.

```

1  -- All arithmetic here lives in Nat – no DvdNat involved.
2  theorem dvd_antisymm_pos
3    {a b : Nat} (ha : 0 < a)
4    (k : Nat) (hk : b = a * k)
5    (j : Nat) (hj : a = b * j) : a = b := by
6    -- Step 1: derive k * j = 1.
7    -- Substitute hk into hj to get a = a * (k * j),
8    -- then cancel a (valid since a > 0).
9    have hkj : k * j = 1 := by
10     have hstep : a = a * (k * j) :=
11       calc a = b * j           := hj
12            _ = (a * k) * j     := by rw [hk]
13            _ = a * (k * j)     := by rw [Nat.mul_assoc]

```



```

14   exact Nat.eq_of_mul_eq_mul_left ha (by rw [Nat.mul_one]; exact hstep.symm)
15   -- Step 2:  $j = 1$  by Lemma 2 (right factor version).
16   have hj1 :  $j = 1$  := mul_eq_one_right hkj
17   -- Step 3: substitute  $j = 1$  into  $hj : a = b * j$  to get  $a = b * 1 = b$ .
18   have h2 :  $a = b * 1$  := by rw [hj1] at hj; exact hj
19   rw [Nat.mul_one] at h2
20   exact h2

```

Why `Nat.eq_of_mul_eq_mul_left`? This says: if $a > 0$ and $a \cdot x = a \cdot y$, then $x = y$. It is multiplication cancellation for $\mathbb{N}$, available in core Lean. The positivity hypothesis is essential: $0 \cdot x = 0 \cdot y$ for all x, y , so cancellation is unsound at zero.

Assembling the instance

With the lemmas in hand, each field is a short delegation.

```

1  instance : PartialOrder DvdNat where
2
3  -- Reflexivity:  $n \mid n$  via witness  $k = 1$ , since  $n.val = n.val * 1$ .
4  le_refl := fun n =>
5    (1, (Nat.mul_one n.val).symm)
6
7  -- Antisymmetry: case-split on whether  $n.val$  is zero or positive,
8  -- then delegate to Lemma 1 or Lemma 3 respectively.
9  le_antisymm := fun n m (k, hk) (j, hj) => by
10    -- Reduce  $n = m$  (DvdNat equality) to  $n.val = m.val$ 
11    cases n; cases m
12    -- Now  $n$  and  $m$  are gone; we have  $a : \text{Nat}$  as the  $.val$  fields.
13    -- The goal is  $\text{DvdNat.mk } a = \text{DvdNat.mk } b$ .
14    -- congr 1 reduces this to  $a = b$ .
15    congr 1
16    rename_i a b
17    cases Nat.eq_zero_or_pos a with
18    | inl ha =>
19      --  $a = 0$ :  $b = 0 * k = 0$  by Lemma 1, and  $a = 0$ , so  $a = b$ .
20      subst ha
21      exact (dvd_zero_left (b) (k, hk)).symm
22    | inr ha =>
23      --  $a > 0$ : use Lemma 3 directly.
24      exact (dvd_antisymm_pos ha k hk j hj).symm
25
26  -- Transitivity: compose witnesses.  $b.val = a.val * k$  and
27  --  $c.val = b.val * j$ , so  $c.val = a.val * (k * j)$ .
28  le_trans := fun a b c (k, hk) (j, hj) =>

```

```
29 (k * j, by rw [hj, hk, Nat.mul_assoc])
```

Reflection

Notice that the structure of the proof exactly mirrors the structure of the mathematics:

- Reflexivity is trivial because the witness $k = 1$ is explicit.
- Transitivity is trivial because multiplying witnesses gives a new witness: $b = a \cdot k$ and $c = b \cdot j$ compose to $c = a \cdot (kj)$.
- Antisymmetry requires real work because it cannot be witnessed directly — it requires us to reason about what witnesses can possibly exist and conclude they must all be 1.

The lemma `mul_eq_one_left` is the mathematical core. Everything else is book-keeping. When reading an unfamiliar proof, it is always worth asking: where is the actual mathematical content? In this case it is entirely in that one lemma.

1.3.3 Sets

Sets (represented as predicates) form a canonical partial order under inclusion.

```
1 -- We represent sets as indicator functions
2 def Set (α : Type) : Type := α → Prop
3
4 -- Subset inclusion
5 def Set.subset {α : Type} (s t : Set α) : Prop :=
6   ∀ (a : α), s a → t a
7
8 instance {α : Type} : OrderBook.PartialOrder (Set α) where
9   le      := Set.subset
10  le_refl := fun s a ha => ha
11  le_antisymm := fun s t h1 h2 =>
12    funext (fun a => propext (h1 a, h2 a))
13  le_trans := fun s t u h1 h2 a ha => h2 a (h1 a ha)
```

This is our first non-trivial Lean proof. Let us read it carefully:

- `le_refl`: given $a \in s$, produce $a \in s$ — trivial.
- `le_antisymm`: $s \subseteq t$ and $t \subseteq s$ imply $s = t$. We use `funext` (function extensionality) and `propext` (propositional extensionality) to conclude $sa \leftrightarrow ta$ for all a , hence $s = t$.
- `le_trans`: chain the inclusions.

1.4 Hasse Diagrams

Posets are best understood visually. A *Hasse diagram* draws elements as nodes and covers as upward edges: we say a is *covered by* b (written $a < b$) if $a < b$ and there is no c with $a < c < b$.

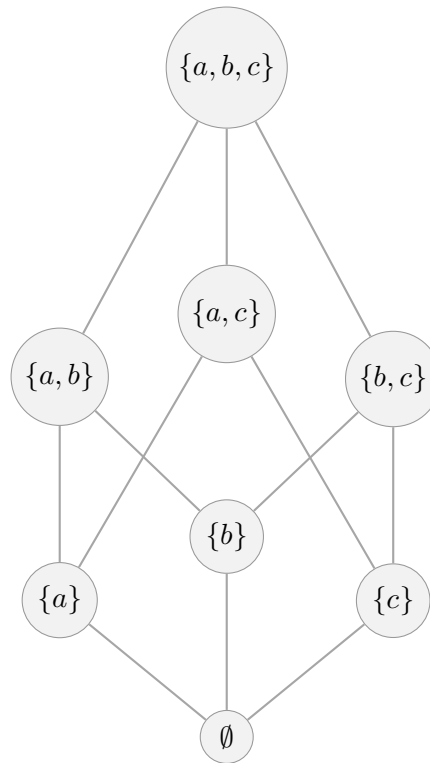
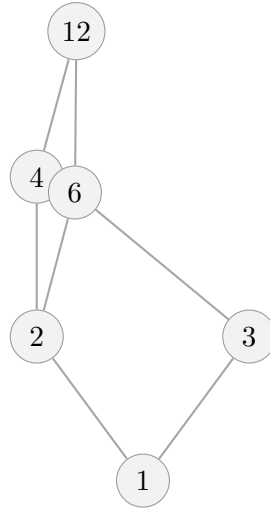


Figure 1.1: The powerset lattice $\mathcal{P}(\{a, b, c\})$ ordered by $\subseteq$.

Figure 1.2: The divisibility poset on $\{1, 2, 3, 4, 6, 12\}$.

1.5 Computation Interlude: Running the Divisibility Order

So far every definition has lived in `Prop`. This section bridges that gap with a fully computable divisibility order on $\{1, \dots, 12\}$.

```

1 def D12 : List Nat := [1, 2, 3, 4, 6, 12]
2 def dvd12 (a b : Nat) : Bool := b % a == 0
3
4 -- Supremum: smallest upper bound
5 def upperBounds (xs : List Nat) : List Nat :=
6   D12.filter (fun u => xs.all (fun x => dvd12 x u))
7
8 def sup12 (xs : List Nat) : Option Nat :=
9   let ub := upperBounds xs
10  ub.find? (fun s => ub.all (fun u => dvd12 s u))
11
12 #eval sup12 [4, 6]   -- some 12 (lcm)
13 #eval sup12 [2, 3]   -- some 6
14
15 -- Infimum: greatest lower bound
16 def lowerBounds (xs : List Nat) : List Nat :=
17   D12.filter (fun l => xs.all (fun x => dvd12 l x))
18
19 def inf12 (xs : List Nat) : Option Nat :=
20   let lbs := lowerBounds xs
21   lbs.find? (fun s => lbs.all (fun l => dvd12 l s))
22
23 #eval inf12 [4, 6]   -- some 2 (gcd)

```

```

24 #eval inf12 [3, 4] -- some 1
25
26 -- Print Hasse diagram edges
27 def covers (a b : Nat) : Bool :=
28   dvd12 a b && a != b &&
29   !D12.any (fun c => c != a && c != b && dvd12 a c && dvd12 c b)
30
31 def printHasse : IO Unit := do
32   for a in D12 do
33     for b in D12 do
34       if covers a b then IO.println s!"{a} → {b}"
35
36 #eval printHasse -- 1→2, 1→3, 2→4, 2→6, 3→6, 4→12, 6→12

```

1.6 Exercises

Exercise 1.1 — Reflexivity and symmetry

Show that a relation that is both symmetric and antisymmetric must be a subset of the equality relation, i.e. $a R b \rightarrow a = b$. Prove this in Lean 4.

```

1 theorem symm_antisymm_eq {α : Type} (r : Rel α)
2   (hs : Symmetric r) (ha : Antisymm r) :
3   ∀ a b : α, r a b → a = b := by
4     intro a b h
5     exact ha a b h (hs a b h)

```

Exercise 1.2 — Divisibility on a small set

Consider the divisibility relation restricted to the set $\{1, 2, 3, 4, 6, 12\}$.

- Draw its Hasse diagram.
- Which pairs of elements are incomparable?
- In Lean 4, define a type `D12` with six constructors and prove that divisibility on it satisfies `le_refl`.

Exercise 1.3 — Product order

Given two partial orders $(A, \leq_A)$ and $(B, \leq_B)$, define the *product order* on $A \times B$ by $(a_1, b_1) \leq (a_2, b_2) \iff a_1 \leq_A a_2 \wedge b_1 \leq_B b_2$. Prove in Lean 4 that this is indeed a partial order by implementing the typeclass instance.

```

1 instance [OrderBook.PartialOrder  $\alpha$ ] [OrderBook.PartialOrder  $\beta$ ] :
2   OrderBook.PartialOrder ( $\alpha \times \beta$ ) where
3   le p q := PartialOrder.le p.1 q.1  $\wedge$  PartialOrder.le p.2 q.2
4   le_refl := fun <a, b> => (PartialOrder.le_refl a,
5                               PartialOrder.le_refl b)
6   le_antisymm := fun <a1,b1> <a2,b2> <h1,h2> <h3,h4> => by
7     have := PartialOrder.le_antisymm _ _ h1 h3
8     have := PartialOrder.le_antisymm _ _ h2 h4
9     simp_all
10  le_trans := fun <a1,b1> <a2,b2> <a3,b3> <h1,h2> <h3,h4> =>
11    (PartialOrder.le_trans _ _ _ h1 h3,
12     PartialOrder.le_trans _ _ _ h2 h4)

```

Exercise 1.4 — Reverse order

If $(A, \leq)$ is a partial order, then so is $(A, \geq)$ where $a \geq b \iff b \leq a$. This is called the *dual* or *opposite* order. Implement a `Dual` wrapper type and its `PartialOrder` instance. We will use this construction crucially when encoding Sudoku domains in Chapter 8.

Chapter 2

Special Elements and Monotone Maps

In a poset, certain elements play privileged roles (least, greatest, bounds), and certain functions are especially well-behaved (monotone maps). This chapter equips us with the vocabulary for both.

2.1 Extremal Elements

Definition — Minimum, Maximum, Minimal, Maximal

Let $(P, \leq)$ be a poset and $S \subseteq P$.

- $m \in S$ is the *minimum* of S if $\forall s \in S, m \leq s$.
- $m \in S$ is the *maximum* of S if $\forall s \in S, s \leq m$.
- $m \in S$ is *minimal* in S if $\nexists s \in S, s < m$.
- $m \in S$ is *maximal* in S if $\nexists s \in S, m < s$.

A minimum, if it exists, is unique (by antisymmetry). A minimal element need not be unique. When a poset has a unique minimum we call it $\perp$ (bottom), and a unique maximum $\top$ (top).

The distinction between minimum and minimal is subtle but important. A *minimum* beats every other element; a *minimal* element merely has no element strictly below it.

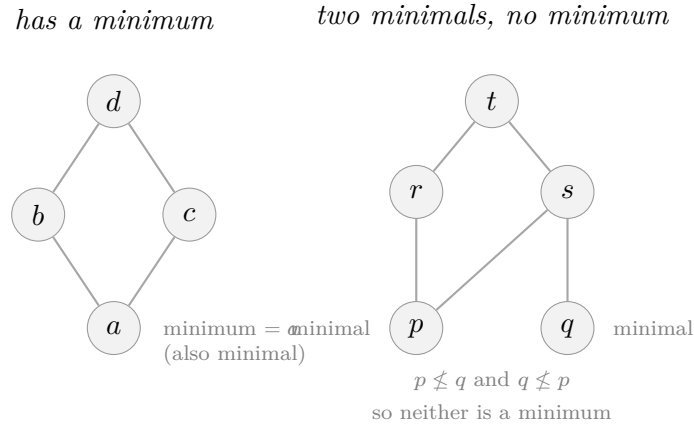


Figure 2.1: Left: a poset with a unique minimum a (every element is above a). Right: a poset where p and q are both minimal (nothing is below either) but there is no minimum (neither $p \leq q$ nor $q \leq p$).

```

1 namespace OrderBook
2
3 -- Bottom and top elements
4 class Bot (α : Type) where bot : α
5 class Top (α : Type) where top : α
6
7 notation "⊥" => Bot.bot
8 notation "⊤" => Top.top
9
10 -- Being a bounded partial order
11 class BoundedPartialOrder (α : Type) extends PartialOrder α, Bot α, Top α
12   ↪ where
13     bot_le : ∀ (a : α), (⊥ : α) ≤ a
14     le_top : ∀ (a : α), a ≤ (⊤ : α)
15 end OrderBook
    
```

Example — Bounded orders

- $(\mathcal{P}(S), \subseteq)$: $\perp = \emptyset$, $\top = S$.
- $(\{0, 1\}, \leq)$: $\perp = 0$, $\top = 1$.
- $(\mathbb{N}, \leq)$: $\perp = 0$, no $\top$ (unbounded above).
- The divisibility poset on $\{1, \dots, 12\}$: $\perp = 1$ (divides everything), no single $\top$.

2.2 Upper and Lower Bounds

Definition — Bounds

Let $(P, \leq)$ be a poset and $S \subseteq P$.

- $u \in P$ is an *upper bound* of S if $\forall s \in S, s \leq u$.
- $l \in P$ is a *lower bound* of S if $\forall s \in S, l \leq s$.
- The *supremum* (or *least upper bound*) of S , written $\sup S$, is the smallest upper bound of S when it exists.
- The *infimum* (or *greatest lower bound*) of S , written $\inf S$, is the largest lower bound of S when it exists.

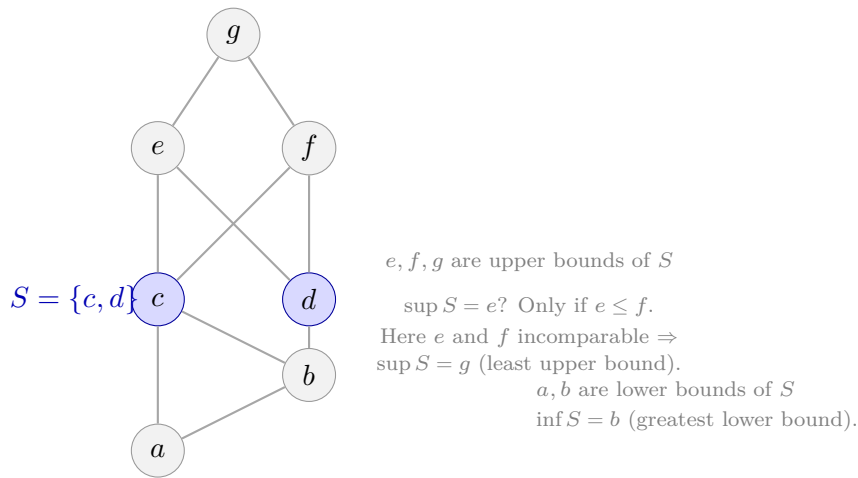


Figure 2.2: Illustration of upper bounds, lower bounds, supremum and infimum for the subset $S = \{c, d\}$ (shaded blue) in a poset. The upper bounds of S are all elements $\geq$ both c and d : here e , f , and g . The *supremum* is the *least* of these; since e and f are incomparable, the least upper bound is g . The lower bounds are a and b ; the infimum is b since $a \leq b$.

Uniqueness of sup/inf

In a poset, if $\sup S$ exists, it is unique. Similarly for $\inf S$.

Proof. Suppose u and v are both least upper bounds of S . Since u is an upper bound and v is a *least* upper bound, $v \leq u$. By symmetry $u \leq v$. Antisymmetry gives $u = v$. $\square$

```

1  -- The supremum of two elements (binary version)
2  -- (We'll generalise to arbitrary subsets in Chapter 4.)
3  structure IsSupOf {α : Type} [OrderBook.PartialOrder α]
4    (a b sup : α) : Prop where
5    ge_a   : a ≤ sup

```

```

6   ge_b  : b ≤ sup
7   least : ∀ (u : α), a ≤ u → b ≤ u → sup ≤ u
8
9   -- Uniqueness of the binary sup
10  theorem sup_unique {α : Type} [OrderBook.PartialOrder α]
11    {a b s t : α}
12    (hs : IsSupOf a b s) (ht : IsSupOf a b t) : s = t := by
13  apply PartialOrder.le_antisymm
14  · exact hs.least t ht.ge_a ht.ge_b
15  · exact ht.least s hs.ge_a hs.ge_b
    
```

2.3 Monotone Maps

Definition — Monotone Map

A function $f : (P, \leq_P) \rightarrow (Q, \leq_Q)$ between posets is *monotone* (or *order-preserving*) if $\forall a, b \in P, a \leq_P b \rightarrow f(a) \leq_Q f(b)$.

Monotone maps are the morphisms of the category of posets. They will be central to propagators: every propagator is a monotone function on a lattice.

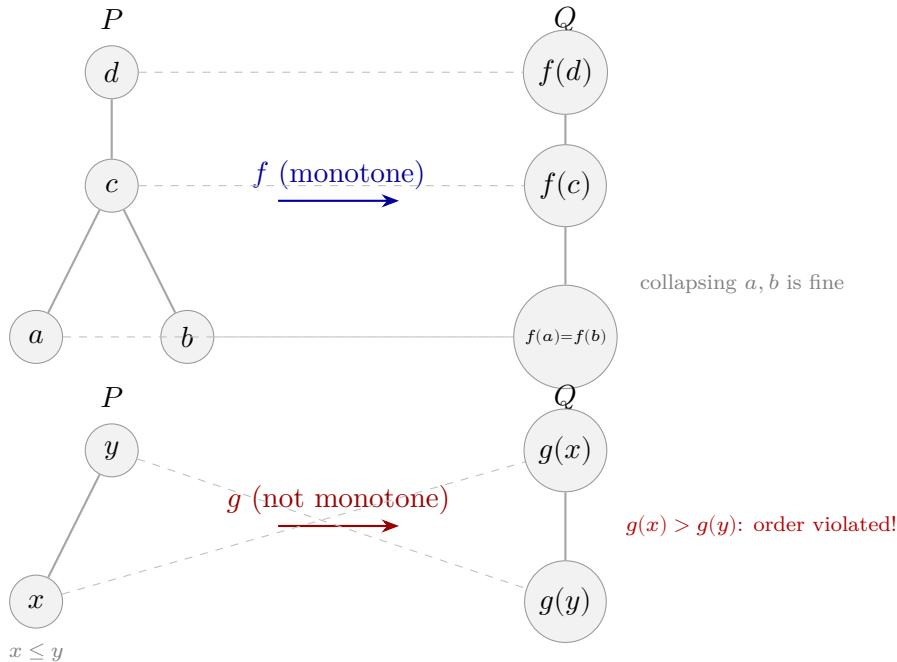


Figure 2.3: Top: a monotone map f preserves the order — whenever $a \leq_P b$ in the source, $f(a) \leq_Q f(b)$ in the target. Here f collapses the incomparable elements a and b to the same image, which is permitted. Bottom: a non-monotone map g where $x \leq y$ but $g(x) > g(y)$, violating the order.

```

1 namespace OrderBook
2
3 structure Monotone {α β : Type}
4   [PartialOrder α] [PartialOrder β] (f : α → β) : Prop where
5     map_le : ∀ (a b : α), a ≤ b → f a ≤ f b
6
7 -- Composition of monotone maps is monotone
8 theorem Monotone.comp {α β γ : Type}
9   [PartialOrder α] [PartialOrder β] [PartialOrder γ]
10  {f : α → β} {g : β → γ}
11  (hf : Monotone f) (hg : Monotone g) : Monotone (g ∘ f) where
12    map_le := fun a b h => hg.map_le _ _ (hf.map_le _ _ h)
13
14 end OrderBook

```

Example — Monotone functions

- $f(n) = n + 5$ is monotone on $(\mathbb{N}, \leq)$.
- Set union $S \mapsto S \cup \{x\}$ is monotone on $(\mathcal{P}(A), \subseteq)$.
- Set intersection $S \mapsto S \cap T$ is monotone on $(\mathcal{P}(A), \subseteq)$ for fixed T .
- Any constant function $f(x) = c$ is monotone.
- The identity function is always monotone.

2.4 Exercises

Exercise 2.1 — Monotone composition

Give an example showing that the composition of two *antitone* maps (where $a \leq b \rightarrow f(b) \leq f(a)$) is monotone.

Exercise 2.2 — Fixed points

A *fixed point* of $f : P \rightarrow P$ is an element x with $f(x) = x$. Show that if f is monotone and $a \leq f(a)$, then the chain $a \leq f(a) \leq f^2(a) \leq \dots$ is indeed a chain (all elements pairwise comparable with the given order).

Exercise 2.3 — Characterising PartialOrder via Monotone

Show that the identity map $\text{id} : (P, \leq) \rightarrow (P, \leq)$ is always monotone, and prove this in Lean 4 using your `Monotone` structure.

2.5 Computation Interlude: Running Monotone Maps

Building on the computable divisibility order from Chapter 1, we can now compute with monotone maps and verify their properties at runtime.

Checking monotonicity

Given a function on `D12`, we can decide whether it is monotone by checking all pairs.

```

1  -- Check all comparable pairs to see if f preserves the order
2  def isMonotone (f : Nat → Nat) : Bool :=
3    D12.all (fun a =>
4      D12.all (fun b =>
5        -- if a | b then f(a) | f(b)
6        !dvd12 a b || dvd12 (f a) (f b)))
7
8  -- The identity is monotone
9  #eval isMonotone id                -- true
10
11 -- Multiplying by 2: if a | b then 2a | 2b
12 #eval isMonotone (· * 2)           -- true
13
14 -- A non-monotone map: send every element to its "complement" 12/n
15 -- 1↦12, 2↦6, 3↦4, 4↦3, 6↦2, 12↦1 – this reverses the order
16 #eval isMonotone (fun n => 12 / n) -- false (it's antitone)
17
18 -- But composing it with itself gives monotone again
19 #eval isMonotone (fun n => 12 / (12 / n)) -- true

```

Computing fixed points

A fixed point of f is an element where $f(x) = x$. We can find them all.

```

1  def fixedPoints (f : Nat → Nat) : List Nat :=
2    D12.filter (fun x => f x == x)
3
4  #eval fixedPoints id                -- [1, 2, 3, 4, 6, 12]
5  #eval fixedPoints (fun _ => 1)      -- [1]
6  #eval D12.filter (fun n => 12 / n == n) -- [] (no fixed points for n ↦ 12/n)

```

Iterating a monotone map

From Exercise 2.2, we know that if f is monotone and $a \leq f(a)$, the chain $a, f(a), f^2(a), \dots$ is ascending. We can watch it converge.

```

1  -- Apply f repeatedly until fixpoint or step limit
2  def iterToFixpoint (f : Nat → Nat) (start : Nat) (limit : Nat) : List Nat :=
3    let rec go (x : Nat) (n : Nat) (acc : List Nat) : List Nat :=
4      if n = 0 then acc.reverse
5      else let y := f x
6           if y == x then (acc.reverse ++ [x])
7           else go y (n - 1) (x :: acc)
8    go start limit []
9
10 -- f(n) = if 2*n ∈ D12 then 2*n else n
11 def double12 (n : Nat) : Nat :=
12   if D12.contains (n * 2) then n * 2 else n
13
14 #eval iterToFixpoint double12 1 10  -- [1, 2, 4, 12] ascending chain!
15 #eval iterToFixpoint double12 3 10  -- [3, 6, 12]
16 #eval iterToFixpoint double12 12 10 -- [12] already a fixed point

```

The output directly illustrates the ascending chain theorem: starting from 1, each application of `double12` moves strictly upward in the divisibility order until reaching the fixed point 12.

Chapter 3

Lattices

A poset in which every pair of elements has both a supremum and an infimum is called a *lattice*. This apparently modest requirement turns out to yield a rich algebraic structure that shows up in logic, topology, computer science, and — as we will see in Part II — concurrency and constraint solving.

3.1 Meet and Join

Definition — Lattice

A *lattice* is a poset $(L, \leq)$ in which every two elements $a, b \in L$ have:

- a *meet* (greatest lower bound) $a \sqcap b$, and
- a *join* (least upper bound) $a \sqcup b$.

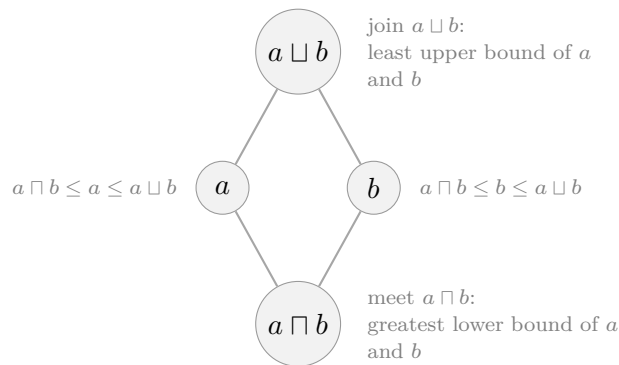


Figure 3.1: The meet $a \sqcap b$ sits below both a and b ; the join $a \sqcup b$ sits above both. In a lattice every pair of elements has both.

The meet and join satisfy the following axioms (which can be taken as an equivalent *algebraic* definition of a lattice):

$a \sqcap b = b \sqcap a$	$a \sqcup b = b \sqcup a$	(commutativity)
$(a \sqcap b) \sqcap c = a \sqcap (b \sqcap c)$	$(a \sqcup b) \sqcup c = a \sqcup (b \sqcup c)$	(associativity)
$a \sqcap a = a$	$a \sqcup a = a$	(idempotence)
$a \sqcap (a \sqcup b) = a$	$a \sqcup (a \sqcap b) = a$	(absorption)

The order relation can be recovered from meet or join:

$$a \leq b \iff a \sqcap b = a \iff a \sqcup b = b.$$

In Lean 4:

```

1 namespace OrderBook
2
3 class Lattice (α : Type) extends PartialOrder α where
4   inf : α → α → α    -- meet (⊓)
5   sup : α → α → α    -- join (⊔)
6   -- meet is a lower bound
7   inf_le_left  : ∀ (a b : α), inf a b ≤ a
8   inf_le_right : ∀ (a b : α), inf a b ≤ b
9   -- meet is the greatest lower bound
10  le_inf : ∀ (a b c : α), a ≤ b → a ≤ c → a ≤ inf b c
11  -- join is an upper bound
12  sup_le_left  : ∀ (a b : α), a ≤ sup a b
13  sup_le_right : ∀ (a b : α), b ≤ sup a b
14  -- join is the least upper bound
15  le_sup : ∀ (a b c : α), a ≤ c → b ≤ c → sup a b ≤ c
16
17  -- Notation
18  infixl:70 " ⊓ " => Lattice.inf
19  infixl:65 " ⊔ " => Lattice.sup
20
21 end OrderBook

```

3.1.1 Derived Properties

We can immediately derive commutativity and idempotence from the axioms:

```

1 namespace OrderBook
2 open Lattice PartialOrder
3
4 variable {α : Type} [Lattice α]
5

```



```

6  -- Commutativity of meet
7  theorem inf_comm (a b :  $\alpha$ ) : a  $\sqcap$  b = b  $\sqcap$  a := by
8    apply le_antisymm
9      · apply le_inf
10       · exact inf_le_right a b
11       · exact inf_le_left a b
12     · apply le_inf
13       · exact inf_le_right b a
14       · exact inf_le_left b a
15
16  -- Idempotence of meet
17  theorem inf_idem (a :  $\alpha$ ) : a  $\sqcap$  a = a := by
18    apply le_antisymm
19      · exact inf_le_left a a
20      · exact le_inf a a a (le_refl a) (le_refl a)
21
22  -- Commutativity of join
23  theorem sup_comm (a b :  $\alpha$ ) : a  $\sqcup$  b = b  $\sqcup$  a := by
24    apply le_antisymm
25      · apply le_sup
26        · exact sup_le_right b a
27        · exact sup_le_left b a
28      · apply le_sup
29        · exact sup_le_right a b
30        · exact sup_le_left a b
31
32  -- Absorption: a  $\sqcup$  (a  $\sqcap$  b) = a
33  theorem sup_inf_self (a b :  $\alpha$ ) : a  $\sqcup$  (a  $\sqcap$  b) = a := by
34    apply le_antisymm
35      · apply le_sup
36        · exact le_refl a
37        · exact inf_le_left a b
38      · exact sup_le_left a (a  $\sqcap$  b)
39
40  end OrderBook

```

3.2 The Duality Principle

Every lattice has a *dual*: swap $\sqcap \leftrightarrow \sqcup$ and $\leq \leftrightarrow \geq$. Any theorem proved in a lattice has a dual theorem for free.

Duality Principle

If φ is a theorem about lattices, then so is the statement obtained by replacing every occurrence of $\sqcap$ with $\sqcup$, $\sqcup$ with $\sqcap$, $\leq$ with $\geq$, $\perp$ with $\top$, and $\top$ with $\perp$.

```

1  structure Dual (α : Type) where
2    val : α
3
4  -- Step 1: flip ≤.  a ≤_dual b means b.val ≤_orig a.val.
5  instance {α : Type} [LE α] : LE (Dual α) where
6    le a b := LE.le (α := α) b.val a.val
7
8  -- Step 2: the dual partial order.
9  instance {α : Type} [LE α] [PartialOrder α] : PartialOrder (Dual α) where
10   le_refl {a}      := PartialOrder.le_refl (α := α) a.val
11   le_antisymm {a b} := fun h1 h2 => by
12     cases a; cases b; congr 1
13     exact PartialOrder.le_antisymm (α := α) h2 h1
14   le_trans {a b c} := fun h1 h2 => by
15     cases a; cases b; cases c
16     exact PartialOrder.le_trans (α := α) h2 h1
17
18  -- Step 3: the dual lattice swaps inf and sup.
19  instance {α : Type} [Lattice α] : Lattice (Dual α) where
20   inf      := fun a b => (Lattice.sup (α := α) a.val b.val)
21   sup      := fun a b => (Lattice.inf (α := α) a.val b.val)
22   inf_le_left := fun a b => Lattice.sup_le_left (α := α) a.val b.val
23   inf_le_right := fun a b => Lattice.sup_le_right (α := α) a.val b.val
24   sup_le_left  := fun a b => Lattice.inf_le_left (α := α) a.val b.val
25   sup_le_right := fun a b => Lattice.inf_le_right (α := α) a.val b.val
26   le_inf := fun a b c h1 h2 => by
27     change Lattice.sup (α := α) b.val c.val ≤ a.val
28     exact Lattice.le_sup (α := α) b.val c.val a.val h1 h2
29   le_sup := fun a b c h1 h2 => by
30     change LE.le (α := α) c.val (Lattice.inf (α := α) a.val b.val)
31     exact Lattice.le_inf (α := α) c.val a.val b.val h1 h2

```

`structure` vs `def`. Using `structure Dual` rather than `def Dual  $\alpha$  :=  $\alpha$` makes `Dual  $\alpha$` and `$\alpha$` genuinely distinct types. With a transparent `def`, Lean’s elaborator sometimes picks up the wrong `LE` instance for hypotheses, causing type mismatches even when the terms are definitionally equal. The `structure` version forces explicit `.val` unwrapping and `{...}` rewrapping, which is more verbose but unambiguous. Why cases in `le\_antisymm` and `le\_trans`? cases `a` destructs `Dual.mk v` into the underlying `v :  $\alpha$` , so subsequent lemmas see plain `$\alpha$` inequalities rather than `Dual`-typed ones. In `le\_antisymm`, `congr 1` further reduces `Dual.mk x = Dual.mk y` to `x = y`.

Why `Nat.zero\_mul` in `le\_antisymm` and `a b : , r a b  $\rightarrow$  a = b := by intro a b h exact`

After destructuring `{a} {b} {c}`, the hypotheses

`structure Monotone { : Type} [PartialOrder ] [PartialOrder ] (f :  $\rightarrow$  ) : Prop where ma`

and `class Lattice ( : Type) extends PartialOrder where inf :  $\rightarrow$   $\rightarrow$  -- meet ( ) sup :  $\rightarrow$`

are still elaborated under the dual `LE`. `Nat.zero\_mul` restates the goal entirely in terms of `$\alpha$` ’s operations, giving Lean enough information to match the original lemma’s type.

Why `LE (Dual  $\alpha$ )` and `le := (  $\cdot$   $\cdot$  ) -- Bool has a built-in LE in core Lean 4 le_refl :`

`swap` is the whole point: turning the Hasse diagram upside down exchanges every meet with every join.

3.3 Important Lattice Examples

3.3.1 The Two-Element Lattice 2

```

1  -- Bool as a lattice: false < true
2  instance : OrderBook.Lattice Bool where
3    le      := (  $\cdot$   $\leq$   $\cdot$  ) -- Bool has a built-in LE in core Lean 4
4    le_refl := Bool.le_refl
5    le_antisymm := Bool.le_antisymm
6    le_trans  := Bool.le_trans
7    inf       := (  $\cdot$  &&  $\cdot$  )
8    sup       := (  $\cdot$  ||  $\cdot$  )
9    inf_le_left  := by decide
10   inf_le_right := by decide
11   le_inf       := by decide
12   sup_le_left  := by decide
13   sup_le_right := by decide
14   le_sup       := by decide

```

The `by decide` tactic works here because `Bool` is a finite type and all the statements are decidable.

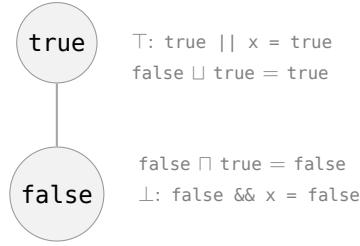


Figure 3.2: The two-element lattice $\mathbf{2}$ on `Bool`. Meet is `&&`, join is `||`.

3.3.2 The Powerset Lattice

```

1  -- Sets ( $\alpha \rightarrow \text{Prop}$ ) form a lattice under subset inclusion
2  def Set.inter { $\alpha$  : Type} (s t : Set  $\alpha$ ) : Set  $\alpha$  := fun a => s a  $\wedge$  t a
3  def Set.union { $\alpha$  : Type} (s t : Set  $\alpha$ ) : Set  $\alpha$  := fun a => s a  $\vee$  t a
4
5  instance { $\alpha$  : Type} : OrderBook.Lattice (Set  $\alpha$ ) where
6    le      := Set.subset
7    le_refl := fun s a ha => ha
8    le_antisymm := fun s t h1 h2 => funext fun a => propext (h1 a, h2 a)
9    le_trans := fun s t u h1 h2 a ha => h2 a (h1 a ha)
10   inf      := Set.inter
11   sup      := Set.union
12   inf_le_left  := fun s t a (hs, _) => hs
13   inf_le_right := fun s t a (_, ht) => ht
14   le_inf       := fun s t u hst hsu a ha => (hst a ha, hsu a ha)
15   sup_le_left  := fun s t a hs => Or.inl hs
16   sup_le_right := fun s t a ht => Or.inr ht
17   le_sup       := fun s t u hsu ht u a h =>
18     h.elim (hsu a) (htu a)
    
```

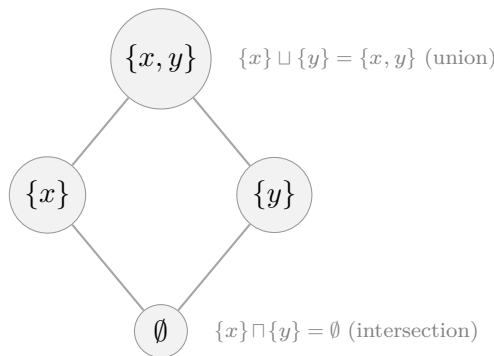


Figure 3.3: The powerset lattice $\mathcal{P}(\{x, y\})$. Meet is $\cap$, join is $\cup$. $\{x\}$ and $\{y\}$ are incomparable.

3.3.3 The Divisibility Lattice on Divisors of n

For divisors of a fixed natural number n , divisibility is a lattice where gcd is the meet and lcm is the join.

```

1  -- On natural numbers: gcd is meet, lcm is join under divisibility order
2  instance : OrderBook.Lattice Nat where
3    le      := (· | ·)  -- divisibility
4    le_refl := Nat.dvd_refl
5    le_antisymm := fun a b => Nat.dvd_antisymm
6    le_trans := fun a b c => Nat.dvd_trans
7    inf     := Nat.gcd
8    sup     := Nat.lcm
9    inf_le_left  := Nat.gcd_dvd_left
10   inf_le_right := Nat.gcd_dvd_right
11   le_inf      := fun a b c => Nat.dvd_gcd
12   sup_le_left  := Nat.dvd_lcm_left
13   sup_le_right := Nat.dvd_lcm_right
14   le_sup      := fun a b c ha hb => Nat.lcm_dvd ha hb

```

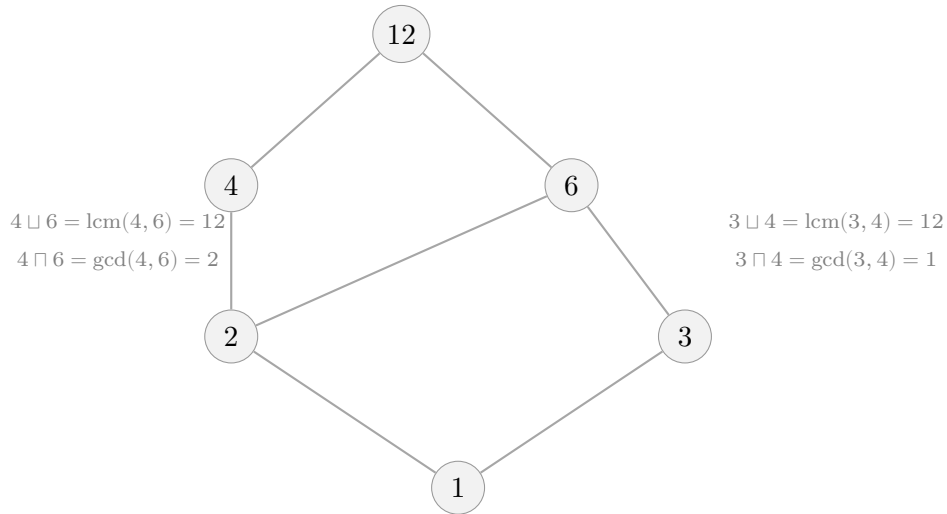


Figure 3.4: The divisibility lattice on divisors of 12. Meet is gcd, join is lcm. Every incomparable pair still has a gcd and lcm in the set, confirming this is a lattice.

3.4 Distributive Lattices

Not every lattice satisfies the distributive law; those that do are especially well-behaved.

Definition — Distributive Lattice

A lattice is *distributive* if $a \sqcap (b \sqcup c) = (a \sqcap b) \sqcup (a \sqcap c)$ (equivalently, the dual law holds too).

The powerset lattice and Boolean lattices are distributive. Figure 3.5 shows the two smallest non-distributive lattices.

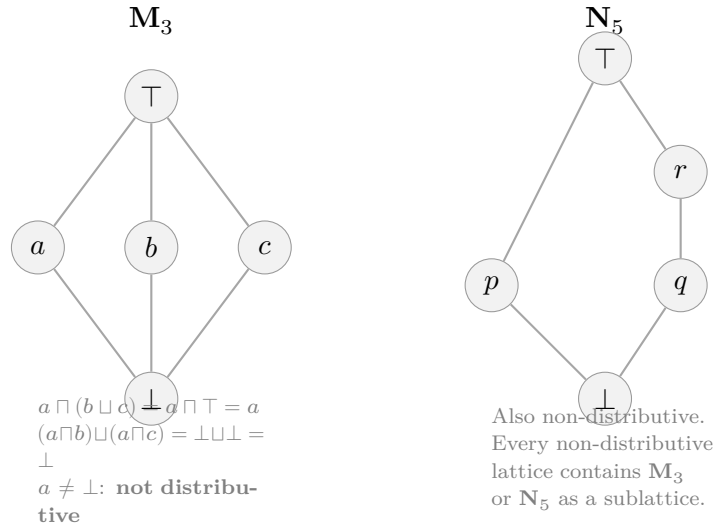


Figure 3.5: The two smallest non-distributive lattices. M_3 (left) has three mutually incomparable middle elements; the distributive law fails because the meet does not distribute over the join. N_5 (right) is the pentagon lattice. A classical theorem states that a lattice is distributive if and only if it contains neither as a sublattice.

3.5 Exercises

Exercise 3.1 — Lattice of intervals

Define the type `Interval` of pairs $[l, h]$ with $l, h : \mathbb{Z}$ and $l \leq h$. Define an order where $[l_1, h_1] \leq [l_2, h_2]$ iff $l_2 \leq l_1$ and $h_1 \leq h_2$ (i.e. “containment by a smaller interval”, the *precision* or *information* order). Prove that this is a partial order in Lean 4. What are the meet and join in this order?

Exercise 3.2 — Absorb, then join

Prove in Lean 4 that in any lattice: $a \sqcap (a \sqcup b) = a$ (absorption).

Exercise 3.3 — Characterising the order

Prove the equivalence $a \leq b \iff a \sqcup b = b$ in Lean 4, using only the lattice axioms.

```

1 theorem le_iff_sup_eq {α : Type} [OrderBook.Lattice α] (a b : α) :
2   a ≤ b ↔ a ⊔ b = b := by
3   constructor
4   · intro h
5     apply PartialOrder.le_antisymm
6     · exact Lattice.le_sup a b b h (PartialOrder.le_refl b)
7     · exact Lattice.sup_le_right a b
8   · intro h
9     have : b = a ⊔ b := h.symm
10    rw [this]
11    exact Lattice.sup_le_left a b

```

Exercise 3.4 — The product lattice

Given two lattices $[Lattice\ \alpha]$ and $[Lattice\ \beta]$, define a $Lattice\ (\alpha \times \beta)$ instance with the component-wise order: $(a_1, b_1) \leq (a_2, b_2) \iff a_1 \leq a_2 \wedge b_1 \leq b_2$, and component-wise meet and join: $(a_1, b_1) \sqcap (a_2, b_2) = (a_1 \sqcap a_2, b_1 \sqcap b_2)$, $(a_1, b_1) \sqcup (a_2, b_2) = (a_1 \sqcup a_2, b_1 \sqcup b_2)$. Prove all eight lattice axioms. (*Hint*: each axiom reduces to the corresponding axiom applied component-wise.)

Chapter 4

Complete Lattices and the Fixed-Point Theorem

Lattices let us take meets and joins of *pairs* of elements. A *complete* lattice extends this to arbitrary (possibly infinite) subsets. This extra power buys us one of the most useful theorems in theoretical computer science: the Knaster–Tarski fixed-point theorem.

4.1 Complete Lattices

Definition — Complete Lattice

A lattice $(L, \leq)$ is *complete* if every subset $S \subseteq L$ has a supremum $\bigvee S$ and an infimum $\bigwedge S$ in L .

In particular, the empty set has a supremum (which becomes $\perp$) and infimum (which becomes $\top$), so every complete lattice is bounded. Taking $S = L$ gives $\bigvee L = \top$ and $\bigwedge L = \perp$.

We encode complete lattices using a predicate representing a subset:

```
1 namespace OrderBook
2
3 -- We represent a "subset" as a predicate  $\alpha \rightarrow \text{Prop}$ .
4 -- This is the same definition as `Set  $\alpha$ ` from Chapter 1
5 -- (`def Set ( $\alpha$  : Type) : Type :=  $\alpha \rightarrow \text{Prop}$ `).
6 -- We use `abbrev` rather than `def` so that Lean treats `Pred  $\alpha$ `
7 -- as completely transparent – unfolding it automatically during
8 -- elaboration without needing `simp` or `unfold`. This makes the
9 -- `sSup`/`sInf` signatures below easier to work with.
10 -- `def Set` was opaque because we wanted `Set  $\alpha$ ` to have its own
11 -- identity for attaching instances; here we just want a readable alias.
12 abbrev Pred ( $\alpha$  : Type) :=  $\alpha \rightarrow \text{Prop}$ 
13
14 class CompleteLattice ( $\alpha$  : Type) extends Lattice  $\alpha$  where
15   sSup : Pred  $\alpha \rightarrow \alpha$     -- supremum of a set ( $\square$ )
```

```

16   sInf : Pred α → α    -- infimum of a set   (□)
17   le_sSup : ∀ (s : Pred α) (a : α), s a → a ≤ sSup s
18   sSup_le : ∀ (s : Pred α) (u : α), (∀ a, s a → a ≤ u) → sSup s ≤ u
19   sInf_le : ∀ (s : Pred α) (a : α), s a → sInf s ≤ a
20   le_sInf : ∀ (s : Pred α) (l : α), (∀ a, s a → l ≤ a) → l ≤ sInf s
21
22   end OrderBook

```

The powerset lattice is the canonical example of a complete lattice: $\bigvee \mathcal{F} = \bigcup \mathcal{F}$ and $\bigwedge \mathcal{F} = \bigcap \mathcal{F}$.

```

1  -- Set α is a complete lattice
2  instance {α : Type} : OrderBook.CompleteLattice (Set α) where
3    -- (inheriting the Lattice instance from Chapter 3)
4    sSup := fun F a => ∃ s : Set α, F s ∧ s a    -- union of a family
5    sInf := fun F a => ∀ s : Set α, F s → s a    -- intersection of a family
6    le_sSup := fun F s hs a ha => ⟨s, hs, ha⟩
7    sSup_le := fun F u hu a ⟨s, hs, ha⟩ => hu s hs a ha
8    sInf_le := fun F s hs a ha => ha s hs
9    le_sInf := fun F l hl a ha s hs => hl s hs a ha
10   -- (le, inf, sup fields inherited)
11   le      := Set.subset
12   le_refl := fun s a h => h
13   le_antisymm := fun s t h1 h2 => funext fun a => propext ⟨h1 a, h2 a⟩
14   le_trans  := fun s t u h1 h2 a ha => h2 a (h1 a ha)
15   inf      := Set.inter; sup := Set.union
16   inf_le_left  := fun s t a ⟨hs, _⟩ => hs
17   inf_le_right := fun s t a ⟨_, ht⟩ => ht
18   le_inf      := fun s t u hst hsu a ha => ⟨hst a ha, hsu a ha⟩
19   sup_le_left  := fun s t a hs => Or.inl hs
20   sup_le_right := fun s t a ht => Or.inr ht
21   le_sup      := fun s t u hsu ht u a h => h.elim (hsu a) (htu a)

```

4.2 Why Fixed Points Matter

A *fixed point* of $f : L \rightarrow L$ is an element x where $f(x) = x$. This is where an iterative process *stabilises*: no further application of f changes anything.

- **Propagator networks.** Cell values stop changing when the propagators reach a fixed point of “apply all propagators once”. That stable state is the least fixed point — the most conservative set of conclusions forced by the constraints, nothing more.

- **Dataflow analysis.** A compiler iterates a flow function until it stabilises. The result is the least fixed point of that function.
- **Denotational semantics.** The meaning of a recursive program is the least fixed point of a semantic functional.
- **Regular expressions.** The set matched by a^* is the least fixed point of $S \mapsto \{\varepsilon\} \cup a \cdot S$.

We will return to the propagator connection in Chapter 6, where network execution terminates at the least fixed point of the combined propagation function.

4.3 The Knaster–Tarski Fixed-Point Theorem

Knaster–Tarski (1955)

Let $(L, \leq)$ be a complete lattice and $f : L \rightarrow L$ a monotone function. Then f has a *least fixed point* $\mu f = \bigwedge \{x \mid f(x) \leq x\}$ and a *greatest fixed point* $\nu f = \bigvee \{x \mid x \leq f(x)\}$.

Proof. Let $P = \{x \in L \mid f(x) \leq x\}$ (the set of *pre-fixed points*) and let $m = \bigwedge P$.

Step 1: $f(m) \leq m$. For every $x \in P$ we have $m \leq x$, so $f(m) \leq f(x) \leq x$ by monotonicity. Hence $f(m)$ is a lower bound of P , so $f(m) \leq m$.

Step 2: $m \leq f(m)$. From Step 1, $f(f(m)) \leq f(m)$ (applying f to $f(m) \leq m$ and using monotonicity gives $f(f(m)) \leq f(m)$, so $f(m) \in P$). Since m is the infimum of P , we have $m \leq f(m)$.

Conclusion: $f(m) = m$, so m is a fixed point. Any other fixed point x^* satisfies $f(x^*) = x^* \leq x^*$, so $x^* \in P$, hence $m \leq x^*$. So m is the *least* fixed point. $\square$

```

1 namespace OrderBook
2 open CompleteLattice PartialOrder
3
4 -- Knaster-Tarski: least fixed point of a monotone function
5 -- on a complete lattice
6 theorem knaster_tarski {α : Type} [CompleteLattice α]
7   (f : α → α) (hf : Monotone f) :
8     ∃ (lfp : α), f lfp = lfp ∧
9       ∀ (x : α), f x = x → lfp ≤ x := by
10   -- Define P = {x | f(x) ≤ x} and m = Ⓜ P
11   let P : Pred α := fun x => f x ≤ x
12   let m := sInf P
13   -- Step 1: f(m) ≤ m
14   have step1 : f m ≤ m := by
15     apply le_sInf

```

```

16   intro x hx
17   -- m ≤ x, so f(m) ≤ f(x) ≤ x
18   exact le_trans (hf.map_le m x (sInf_le P x hx)) hx
19   -- Step 2: m ≤ f(m) (f(m) ∈ P since f(f(m)) ≤ f(m))
20   have step2 : m ≤ f m := by
21     apply sInf_le
22     exact hf.map_le _ _ step1
23   -- Conclude m is a fixed point
24   have fixed : f m = m := le_antisymm step1 step2
25   refine ⟨m, fixed, ?_⟩
26   -- Minimality
27   intro x hx
28   apply sInf_le
29   show f x ≤ x
30   exact le_of_eq hx
31 end OrderBook
    
```

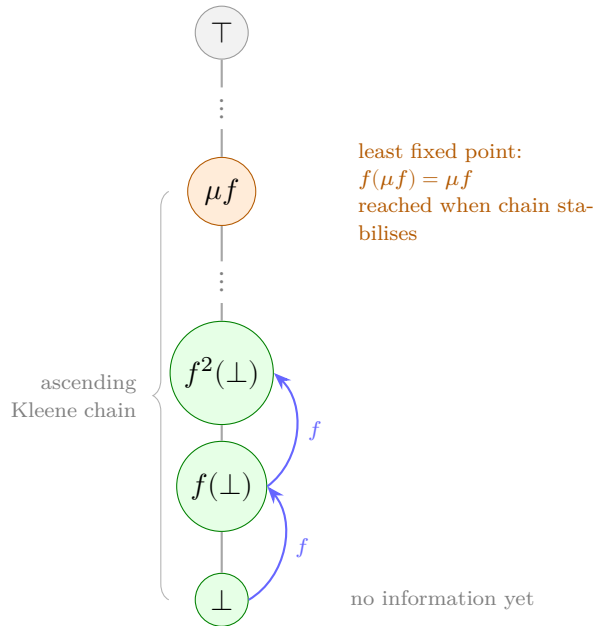


Figure 4.1: The ascending Kleene chain starting from $\perp$ under a monotone function f . Each application of f moves upward. If the lattice satisfies the ascending chain condition the sequence stabilises at the least fixed point μf .

4.4 Chains and Convergence

Chains are the lattice-theoretic version of sequences in calculus, and understanding them is essential for understanding why iterative computations terminate.

Definition — Chain

A *chain* in a poset $(P, \leq)$ is a subset $C \subseteq P$ where every pair of elements is comparable: for all $a, b \in C$, either $a \leq b$ or $b \leq a$. An *ascending chain* is a chain written as a sequence $a_0 \leq a_1 \leq a_2 \leq \dots$.

The parallel with calculus sequences is direct:

Calculus	Lattice theory
Sequence $a_0 \leq a_1 \leq a_2 \leq \dots$	Ascending chain
Limit $\lim_{n \rightarrow \infty} a_n$	Supremum $\bigsqcup_n a_n$
Convergence	Chain stabilising
Completeness of $\mathbb{R}$	Completeness of the lattice
Cauchy sequence	Ascending chain condition

Just as the completeness of $\mathbb{R}$ guarantees that every bounded ascending sequence has a limit, the completeness of a lattice guarantees that every chain has a supremum. Without completeness, the supremum might not exist in your structure — exactly as $\sqrt{2}$ does not exist in $\mathbb{Q}$ even though there are rational sequences converging to it.

The Ascending Chain Condition

A poset satisfies the *ascending chain condition* (ACC) if every strictly ascending chain $a_0 < a_1 < a_2 < \dots$ is finite — there are no infinite strictly ascending sequences.

This is the lattice analogue of a sequence that reaches its limit in finite steps rather than merely approaching it. When the ACC holds, the Kleene chain

$$\perp \leq f(\perp) \leq f^2(\perp) \leq \dots$$

must eventually stabilise at some $f^n(\perp) = f^{n+1}(\perp)$, giving a fixed point in finite iterations.

Example — ACC in practice

FlatNat satisfies the ACC: the only strictly ascending chains are $\perp < \text{known}(n)$, which have length 2. So any propagator network using **FlatNat** cells terminates after at most two updates per cell.

The interval lattice does *not* satisfy the ACC in general: you can have $[0, 10] < [0, 9] < [0, 8] < \dots$ (an infinite strictly descending chain in the information order, equivalently an infinite strictly ascending chain in the reverse order). This is why interval propagators can require a widening operator to force termination.

 $\perp$ is the starting point, not the limit

A common point of confusion: the supremum of the Kleene chain is *not* $\perp$. $\perp$ is the *minimum* of the chain — the smallest element in it. The supremum is the smallest

element above the whole chain, which is wherever the iteration converges.

In the calculus analogy: for the sequence $0, \frac{1}{2}, \frac{3}{4}, \frac{7}{8}, \dots$ converging to 1, the minimum of the sequence is 0 but the supremum (limit) is 1. The starting point and the limit are different objects. The Kleene chain behaves the same way: $\perp$ is where you start, μf is where you end up.

4.5 Exercises

Exercise 4.1 — Greatest fixed point

Using the Knaster–Tarski proof as a template, prove the existence of the *greatest* fixed point $\nu f = \bigvee \{x \mid x \leq f(x)\}$.

Exercise 4.2 — Ascending Kleene chain

For a complete lattice with a bottom element $\perp$ and a monotone f , the sequence $\perp \leq f(\perp) \leq f^2(\perp) \leq \dots$ is called the *ascending Kleene chain*. If the lattice satisfies the *ascending chain condition* (every ascending chain eventually stabilises), this chain reaches μf in finitely many steps. Prove in Lean 4 that each element of this sequence is $\leq$ the next.

Part II

Propagator Networks

Chapter 5

The Propagator Model

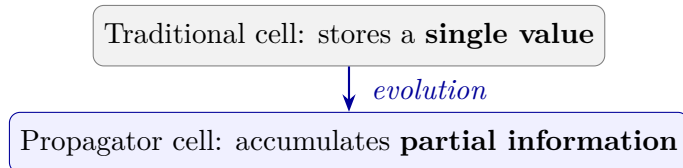
“A propagator network is a collection of machines that compute values for cells by combining information from other cells.”

— G.J. Sussman & A. Radul, *The Art of the Propagator*, 2009

We now have all the mathematical machinery we need. This chapter explains the propagator model from first principles and makes precise why lattices are not merely convenient but *necessary*.

5.1 Cells: Accumulating Partial Information

The central insight of propagator networks is a shift in perspective:



A cell does not *store a value*; it stores *everything we know so far* about a value. New information can arrive from multiple sources and be merged in. The key requirement is that information can only *increase*: once we know something, we cannot un-know it.

This is precisely the structure of a *join-semilattice with a bottom element*: the bottom represents “we know nothing”, and merging two pieces of information is the join ($\sqcup$).

Definition — Information Order

A *bounded join-semilattice* is a partial order $(J, \leq)$ equipped with a join $\sqcup$ and a bottom $\perp$ such that:

- $a \sqcup b$ is the least upper bound of a and b ,
- $\perp \leq a$ for all a ,
- $a \leq b$ is read as “ b contains at least as much information as a ”.

We call this the *information order*.

```
1 namespace Propagators
2
```

```

3 class BoundedJoinSemilattice (α : Type) extends OrderBook.PartialOrder α where
4   sup : α → α → α
5   bot : α
6   le_sup_left  : ∀ (a b : α), a ≤ sup a b
7   le_sup_right : ∀ (a b : α), b ≤ sup a b
8   sup_le       : ∀ (a b c : α), a ≤ c → b ≤ c → sup a b ≤ c
9   bot_le       : ∀ (a : α), bot ≤ a
10
11 -- Convenient notation
12 infixl:65 " ⊔ " => BoundedJoinSemilattice.sup
13 notation    "⊥"  => BoundedJoinSemilattice.bot
14
15 end Propagators

```

5.2 Why Information Can Only Increase

Suppose a propagator computes that “ x is between 3 and 7”. Later another propagator deduces that “ x is between 5 and 12”. Combining these we know “ x is between 5 and 7”. Notice: the combined knowledge is *strictly more specific* than either input.

If we allowed information to *decrease* (e.g., forgetting the $[3, 7]$ constraint and keeping only $[5, 12]$), we would lose soundness: our cell might claim values that the original propagators had ruled out.

Monotonicity Ensures Soundness

In a propagator network over a bounded join-semilattice, if every propagator is monotone, then the network is *sound*: the final cell values are consistent with all constraints.

Intuitively: since each step can only add information (move up in the lattice), the final state contains at least as much information as any intermediate state. Nothing is lost; nothing false is added (assuming correct propagators).

5.3 Propagators as Monotone Functions

Definition — Propagator

Given cells with values in $(J, \leq)$, a *propagator* is a monotone function $f : J^n \rightarrow J$ (reading from n input cells and writing to one output cell by merging the result).

Writing is done by merging: the output cell c receives $c \leftarrow c \sqcup f(\text{inputs})$. Since $\sqcup$ is the least upper bound, this can only increase c .

```

1 namespace Propagators
2
3 -- A unary propagator: a pure function with a bundled monotonicity proof.
4 -- The condition  $\text{fn } a \leq \text{fn } b$  is the standard order-theoretic definition.
5 -- (Equivalent to BoundedJoinSemilattice.sup (fn a) (fn b) = fn b,
6 -- but the  $\leq$  form is cleaner to state and prove.)
7 structure Propagator1 ( $\alpha \beta$  : Type)
8   [BoundedJoinSemilattice  $\alpha$ ]
9   [BoundedJoinSemilattice  $\beta$ ] where
10   fn      :  $\alpha \rightarrow \beta$ 
11   monotone :  $\forall (a \ b : \alpha), a \leq b \rightarrow \text{fn } a \leq \text{fn } b$ 
12
13 -- A binary propagator: monotone in both arguments.
14 structure Propagator2 ( $\alpha \beta \gamma$  : Type)
15   [BoundedJoinSemilattice  $\alpha$ ]
16   [BoundedJoinSemilattice  $\beta$ ]
17   [BoundedJoinSemilattice  $\gamma$ ] where
18   fn      :  $\alpha \rightarrow \beta \rightarrow \gamma$ 
19   monotone :  $\forall (a1 \ a2 : \alpha) (b1 \ b2 : \beta),$ 
20      $a1 \leq a2 \rightarrow b1 \leq b2 \rightarrow \text{fn } a1 \ b1 \leq \text{fn } a2 \ b2$ 
21
22 end Propagators

```

Specification vs. implementation. `Propagator1` and `Propagator2` capture what a propagator *is* mathematically: a function with a bundled monotonicity proof. They are the *specification*.

In practice we use `PropStep = IO Bool`: a plain `IO` action that reads cells, computes, writes, and returns whether anything changed. There is no bundled proof — the user is trusted to write monotone propagators. Section 6.4 below proves that `addProp` and `subProp` do in fact satisfy the `Propagator2` specification. For the capstone projects the proofs would be analogous but verbose, so we omit them.

5.4 Networks, Scheduling, and Quiescence

A *propagator network* is a directed graph where nodes are cells and edges are propagators. Execution proceeds as follows:

1. Initialise all cells to $\perp$.
2. Place all propagators on a *work queue*.
3. While the queue is non-empty:
 - (a) Take a propagator p from the queue.
 - (b) Compute the new value $v = p(\text{input cells})$.

- (c) Merge v into the output cell: $c \leftarrow c \sqcup v$.
 - (d) If c changed, add all propagators that read from c back to the queue.
4. The network has reached *quiescence* when the queue is empty.

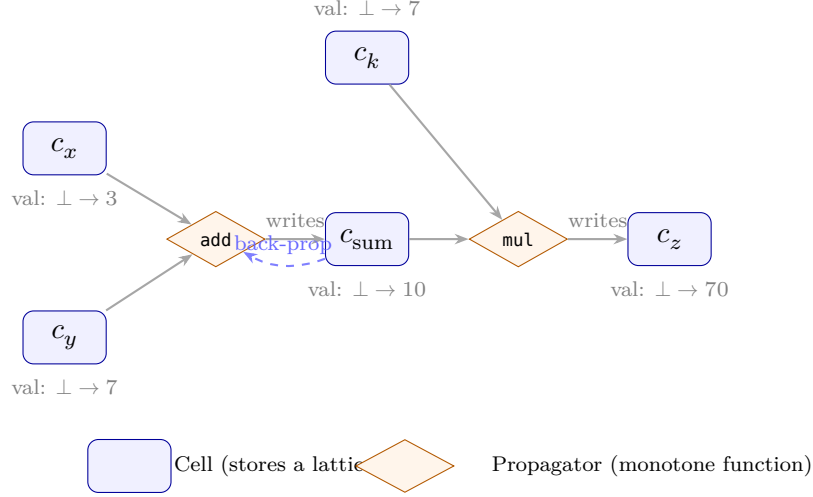


Figure 5.1: A small propagator network computing $x + y = \text{sum}$ and $\text{sum} \times k = z$ (with $x = 3$, $y = 7$, $k = 7$). Cells hold lattice values initialised to $\perp$. Propagators read from inputs and merge results into output cells via $\sqcup$. The dashed arrow shows back-propagation: when c_{sum} is updated the add propagator is re-queued.

Convergence

If the lattice satisfies the *ascending chain condition* (every strictly ascending sequence is finite), then the network reaches quiescence in finite time.

This is immediate: each cell value is non-decreasing and the chain condition guarantees it cannot increase indefinitely.

For our capstone projects, the domain lattice is always finite, so convergence is guaranteed.

5.5 The Fundamental Role of the Lattice

Let us be precise about why we need a lattice rather than merely some arbitrary type.

- **Join ($\sqcup$) for merging.** When two propagators independently derive information about the same cell, we need to combine their results. The join gives the *least informative* value that is *at least as informative* as both. No information is lost; no information is invented.
- **Bottom ($\perp$) for initialisation.** Before any propagator has run, a cell contains *no information*. The bottom element represents this “empty” state.
- **Partial order ($\leq$) for monitoring change.** We must know whether a cell’s

value actually changed after merging, to decide whether to re-schedule dependent propagators. The partial order provides this: if $c \sqcup v = c$ (the new value adds no information), nothing changed.

- **Antisymmetry for termination.** If a cell's value can only go up and the lattice is finite, the antisymmetry of $\leq$ ensures each value is visited at most once.

Any type that fails to be a join-semilattice with $\perp$ is simply not a valid cell domain for a propagator network. The lattice structure is not a convenience — it is a prerequisite.

Chapter 6

Building a Propagator Network in Lean 4

Theory in hand, we now build a complete, runnable propagator network in Lean 4, starting from our typeclass definitions and ending with a working scheduler.

6.1 Mutable Cells

Lean 4 has an `IO` monad with mutable references (`IO.Ref`). We use these to implement cells.

```
1 namespace Propagators
2
3 -- A Cell wraps an IO.Ref and a type with a BoundedJoinSemilattice
4 structure Cell (α : Type) where
5   ref      : IO.Ref α
6
7 variable {α : Type} [BoundedJoinSemilattice α] [DecidableEq α]
8
9 -- Create a new cell initialised to ⊥
10 def Cell.new : IO (Cell α) := do
11   let r ← IO.mkRef (BoundedJoinSemilattice.bot (α := α))
12   return { ref := r }
13
14 -- Read the current value
15 def Cell.read (c : Cell α) : IO α := c.ref.get
16
17 -- Attempt to write new information.
18 -- Returns true if the cell's value increased.
19 def Cell.write (c : Cell α) (v : α) : IO Bool := do
20   let old ← c.ref.get
21   let merged := old ⊔ v
22   if merged == old then
23     return false      -- no new information
24   else
25     c.ref.set merged
```

```

26     return true           -- cell updated, re-schedule dependents
27
28 end Propagators

```

We require `DecidableEq  $\alpha$` to compare `merged == old`. For the finite lattices we use in our capstone projects this is always available.

6.2 The Network and Scheduler

```

1 namespace Propagators
2
3 -- A propagator step is an IO action that returns
4 -- true if it caused any cell to change.
5 abbrev PropStep := IO Bool
6
7 -- Run a list of propagators to fixpoint.
8 -- Each pass re-runs all propagators; we stop when
9 -- a full pass causes no changes.
10 def runToFixpoint (steps : Array PropStep) : IO Unit := do
11   let mut changed := true
12   while changed do
13     changed := false
14     for step in steps do
15       let c ← step
16       if c then changed := true
17
18 -- A smarter scheduler that only re-runs propagators
19 -- whose inputs changed (using a dependency queue).
20 structure Network where
21   steps      : Array PropStep
22   -- In a production implementation, steps would be indexed
23   -- and cells would hold lists of dependent propagator IDs.
24   -- For clarity we use the simple "run all" scheduler here.
25
26 def Network.run (n : Network) : IO Unit :=
27   runToFixpoint n.steps
28
29 end Propagators

```


6.3 Building Propagators: Numeric Example

Let us define the *flat natural number lattice*: information is either “unknown” ($\perp$), a specific number, or “contradiction” ($\top$).

```

1 namespace Propagators
2
3 inductive FlatNat where
4   | unknown      : FlatNat      --  $\perp$ : no information
5   | known (n : Nat) : FlatNat    -- exactly  $n$ 
6   | conflict     : FlatNat      --  $\top$ : contradiction
7
8 instance : OrderBook.PartialOrder FlatNat where
9   le a b := match a, b with
10    | .unknown, _      => True      --  $\text{unknown} \leq \text{everything}$ 
11    | _, .conflict     => True      --  $\text{everything} \leq \text{conflict}$ 
12    | .known m, .known n => m = n  --  $\text{known values only} \leq \text{themselves}$ 
13    | .conflict, .unknown => False
14    | .conflict, .known _ => False
15    | .known _, .unknown  => False
16   le_refl := by intro a; cases a <;> simp
17   le_antisymm := by
18     intro a b hab hba
19     cases a <;> cases b <;> simp_all
20   le_trans := by
21     intro a b c hab hbc
22     cases a <;> cases b <;> cases c <;> simp_all
23
24 instance : BoundedJoinSemilattice FlatNat where
25   le := OrderBook.PartialOrder.le
26   le_refl := OrderBook.PartialOrder.le_refl
27   le_antisymm := OrderBook.PartialOrder.le_antisymm
28   le_trans := OrderBook.PartialOrder.le_trans
29   bot := .unknown
30   sup a b := match a, b with
31    | .unknown, x      => x
32    | x, .unknown      => x
33    | .known m, .known n => if m = n then .known m else .conflict
34    | _, _             => .conflict
35   bot_le := by intro a; cases a <;> simp [OrderBook.PartialOrder.le]
36   le_sup_left := by
37     intro a b
38     cases a <;> cases b <;> simp_all [OrderBook.PartialOrder.le]
39     rename_i m n
40     by_cases h : m = n <;> simp [OrderBook.PartialOrder.le, h]
41   le_sup_right := by

```

```

42   intro a b
43   cases a <=> cases b <=> simp_all [OrderBook.PartialOrder.le]
44   rename_i m n
45   by_cases h : m = n <=> simp [OrderBook.PartialOrder.le, h]
46   sup_le      := by
47     intro a b c hac hbc
48     cases a <=> cases b <=> cases c <=> simp_all [OrderBook.PartialOrder.le]
49
50   deriving instance DecidableEq for FlatNat
51
52   end Propagators

```

6.4 Monotonicity Proofs for addProp and subProp

The `PropStep` actions `addProp` and `subProp` implement pure functions on `FlatNat`. We can extract those functions and prove they satisfy the `Propagator2` specification.

Why conflict propagates. There is a subtlety: the pure function must return `conflict` (not `unknown`) when an input is `conflict`, otherwise monotonicity fails. If we returned `unknown` for conflicted inputs, then `known 3 ≤ conflict` would hold in `FlatNat` but `addFn (known 3) (known 2) = known 5 \not≤ unknown = addFn conflict x`, violating monotonicity. Returning `conflict` fixes this: `known 5 ≤ conflict` always holds.

```

1   namespace Propagators
2
3   -- Pure addition: both inputs known → known sum.
4   -- Either input conflicted → conflict propagates.
5   -- Otherwise → unknown (no information yet).
6   def FlatNat.addFn : FlatNat → FlatNat → FlatNat
7   | .known a, .known b => .known (a + b)
8   | .conflict, _       => .conflict
9   | _, .conflict       => .conflict
10  | _, _               => .unknown
11
12  -- Pure subtraction: known total - known a, with conflict on underflow.
13  def FlatNat.subFn (s x : FlatNat) : FlatNat :=
14    match s, x with
15    | .known total, .known a =>
16      if total ≥ a then .known (total - a) else .conflict
17    | .conflict, _ | _, .conflict => .conflict
18    | _, _                     => .unknown
19
20  -- Proof: addFn is monotone.

```

```

21 -- Strategy: case-split all 81 constructor combinations (3 × 3 × 3 × 3).
22 -- Most cases are trivial:
23 --   • unknown ≤ anything → addFn unknown ... = unknown ≤ anything.
24 --   • anything ≤ conflict → addFn ... conflict = conflict, and anything ≤
    ↪ conflict.
25 --   • known m ≤ unknown is False → hypothesis contradiction closes the goal.
26 -- The only non-trivial case is known m / known n where ha : m = n forces m =
    ↪ n,
27 -- and known p / known q where hb : p = q forces p = q, so both sides are
    ↪ equal.
28 def addPropagator : Propagator2 FlatNat FlatNat FlatNat where
29   fn      := FlatNat.addFn
30   monotone := by
31     intro a1 a2 b1 b2 ha hb
32     cases a1 <;> cases a2 <;> cases b1 <;> cases b2
33     <;> simp_all [FlatNat.addFn, OrderBook.PartialOrder.le]
34
35 -- Proof: subFn is monotone.
36 -- Same strategy. The known/known/known/known case reduces to
37 -- (if m ≥ p ...) ≤ (if m ≥ p ...) since ha : m = m' and hb : p = p'
38 -- force both sides to be identical, which is le_refl.
39 def subPropagator : Propagator2 FlatNat FlatNat FlatNat where
40   fn      := FlatNat.subFn
41   monotone := by
42     intro a1 a2 b1 b2 ha hb
43     cases a1 <;> cases a2 <;> cases b1 <;> cases b2
44     <;> simp_all [FlatNat.subFn, OrderBook.PartialOrder.le]
45     rename_i m n
46     by_cases h : m ≥ n <;> simp [OrderBook.PartialOrder.le, h]
47
48 -- Connection to PropStep actions:
49 -- addProp cx cy csum is addPropagator.fn wrapped in IO:
50 --   it reads cx and cy, computes addFn, and writes to csum.
51 -- subProp csum cx cy implements subPropagator.fn similarly.
52 -- The Propagator2 instances certify that these IO actions are sound.
53
54 end Propagators

```

6.5 A Complete Running Example

Example — Arithmetic propagation

We know: $x + y = 10$ and $x = 3$. A propagator should deduce $y = 7$.

```

1 namespace Propagators
2
3 open FlatNat
4
5 -- Addition propagator: given x and y cells, write x+y to sum cell
6 def addProp (cx cy csum : Cell FlatNat) : PropStep := do
7   let x ← cx.read
8   let y ← cy.read
9   match x, y with
10  | .known a, .known b => csum.write (.known (a + b))
11  | _, _ => return false
12
13 -- Subtraction (used to deduce y = sum - x)
14 def subProp (csum cx cy : Cell FlatNat) : PropStep := do
15   let s ← csum.read
16   let x ← cx.read
17   match s, x with
18   | .known total, .known a =>
19     if total ≥ a
20     then cy.write (.known (total - a))
21     else cy.write .conflict
22   | _, _ => return false
23
24 def exampleNetwork : IO Unit := do
25   let cx ← Cell.new (α := FlatNat)
26   let cy ← Cell.new (α := FlatNat)
27   let csum ← Cell.new (α := FlatNat)
28
29   -- Seed known facts
30   let _ ← cx.write (.known 3)
31   let _ ← csum.write (.known 10)
32
33   let net : Network := {
34     steps := #[
35       addProp cx cy csum,
36       subProp csum cx cy,
37       subProp csum cy cx
38     ]
39   }
40
```

```

41 net.run
42
43 let yVal ← cy.read
44 match yVal with
45 | .known n => IO.println s!"y = {n}"    -- prints: y = 7
46 | .unknown => IO.println "y is unknown"
47 | .conflict => IO.println "contradiction!"
48
49 end Propagators

```

6.6 Why the Lattice Is Load-Bearing Here

Notice several things:

- `Cell.write` can be called multiple times from different propagators without coordination. The join ensures the final value is consistent with all of them.
- Running `addProp` when either input is `unknown` correctly does nothing (returns `false`), waiting for more information.
- The `conflict` element propagates correctly: if two propagators derive incompatible values, the join produces `conflict`, which then propagates to any cell that depends on this one.
- The whole system terminates because `FlatNat` has height 2: after at most two increases, any cell stabilises.

6.7 Exercises

Exercise 6.1 — Three-cell constraint

Add a `mulProp` propagator for $x \times y = z$ with the `FlatNat` lattice. Build a network with $x \times y = 12$, $x = 3$, and verify it deduces $y = 4$.

Exercise 6.2 — Conflict detection

Seed a network with $x = 3$ from one source and $x = 5$ from another. Verify that `cx.read` returns `.conflict`.

Chapter 7

The Interval Lattice

Flat lattices are powerful but coarse: a cell either knows the exact value or knows nothing. *Interval lattices* let us represent partial numeric knowledge as a range, enabling *bound propagation* — the engine behind many real-world constraint solvers.

7.1 Intervals as Partial Information

An interval $[l, h]$ says: “the true value lies between l and h ”. The wider the interval, the less we know. The narrowest possible interval is a single point $[n, n]$, meaning we know the value exactly.

Definition — Interval Lattice

Let the domain be extended integers $\mathbb{Z}_\infty = \mathbb{Z} \cup \{-\infty, +\infty\}$. Define:

- An *interval* is either $\perp$ (no values possible, contradiction) or $[l, h]$ with $l \leq h$.
- The *information order*: $[l_1, h_1] \leq [l_2, h_2] \iff l_2 \leq l_1 \wedge h_1 \leq h_2$ (a narrower interval contains more information).
- $\perp \leq [l, h]$ for all intervals.
- The *join* (merge two pieces of information): $[l_1, h_1] \sqcup [l_2, h_2] = [\min(l_1, l_2), \max(h_1, h_2)]$ (the smallest interval containing both).
- $\perp$ is the identity for $\sqcup$.

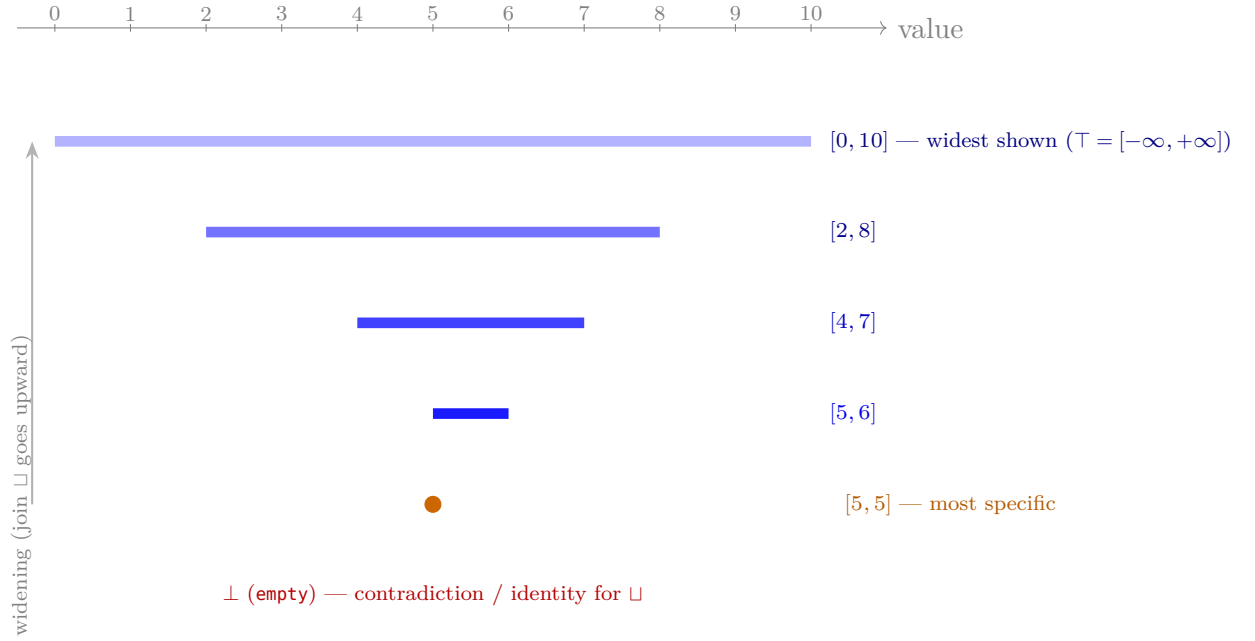


Figure 7.1: The interval lattice order. $\perp$ sits at the bottom as the join identity. Going upward, intervals widen: the join (`Interval.sup`) produces the hull (smallest interval containing both), which is at least as wide as either input. The meet (`Interval.intersect`) produces the intersection, which is narrower and thus lower.

```

1 namespace Intervals
2
3 inductive Interval where
4   | empty          : Interval  -- ⊥
5   | range (lo hi : Int) (h : lo ≤ hi) : Interval
6
7 -- Information order: narrower = more informative = higher
8 def Interval.le (a b : Interval) : Prop :=
9   match a, b with
10  | .empty, _           => True
11  | .range _ _ _, .empty => False
12  | .range l1 h1 _, .range l2 h2 _ => l2 ≤ l1 ∧ h1 ≤ h2
13
14 -- Join: widen to contain both intervals
15 def Interval.sup (a b : Interval) : Interval :=
16   match a, b with
17   | .empty, x => x
18   | x, .empty => x
19   | .range l1 h1 _, .range l2 h2 _ =>
20     let l := min l1 l2
21     let h := max h1 h2
22     .range l h (by omega)
23

```



```

24 instance : OrderBook.PartialOrder Interval where
25   le      := Interval.le
26   le_refl  := by intro a; cases a <;> simp [Interval.le]
27   le_antisymm := by
28     intro a b hab hba
29     cases a <;> cases b <;>
30     simp [Interval.le] at * <;> omega
31   le_trans  := by
32     intro a b c hab hbc
33     cases a <;> cases b <;> cases c <;>
34     simp [Interval.le] at * <;> omega
35
36 -- BoundedJoinSemilattice: ⊥ = empty, join = hull (Interval.sup)
37 instance : Propagators.BoundedJoinSemilattice Interval where
38   le      := Interval.le
39   le_refl  := OrderBook.PartialOrder.le_refl
40   le_antisymm := OrderBook.PartialOrder.le_antisymm
41   le_trans  := OrderBook.PartialOrder.le_trans
42   bot      := .empty
43   sup      := Interval.sup
44   bot_le    := by intro a; cases a <;> simp [Interval.le]
45   le_sup_left := by
46     intro a b; cases a <;> cases b <;> simp [Interval.sup, Interval.le]
47     exact ⟨Int.min_le_left _ _, Int.le_max_left _ _⟩
48   le_sup_right := by
49     intro a b; cases a <;> cases b <;> simp [Interval.sup, Interval.le]
50     exact ⟨Int.min_le_right _ _, Int.le_max_right _ _⟩
51   sup_le      := by
52     intro a b c hac hbc
53     cases a <;> cases b <;> cases c <;>
54     simp [Interval.sup, Interval.le] at * <;> omega
55
56 -- DecidableEq needed by Cell.write
57 instance : DecidableEq Interval := by
58   intro a b
59   cases a <;> cases b
60   · exact isTrue rfl
61   · exact isFalse (by simp)
62   · exact isFalse (by simp)
63   · rename_i l1 h1 _ l2 h2 _
64     by_cases hl : l1 = l2
65     · by_cases hh : h1 = h2
66       · subst hl; subst hh; exact isTrue (by congr 1; exact Subsingleton.elim
67         ↪ _ _)
68       · exact isFalse (by intro heq; cases heq; exact hh rfl)
69     · exact isFalse (by intro heq; cases heq; exact hl rfl)

```

69

70 `end Intervals`

7.2 Interval Arithmetic Propagators

To use `Interval` with the `Cell` infrastructure from Chapter 6, we need two instances: `BoundedJoinSemilattice Interval` (so the scheduler can merge values with $\sqcup$ and initialise cells to $\perp$) and `DecidableEq Interval` (so `Cell.write` can detect whether the value actually changed). Both are provided above.

With the interval lattice, we can implement real *bound propagation*.

```
1 namespace Intervals
2
3 -- Add two intervals
4 def Interval.add (a b : Interval) : Interval :=
5   match a, b with
6   | .empty, _ | _, .empty => .empty
7   | .range l1 h1 _, .range l2 h2 _ =>
8     .range (l1 + l2) (h1 + h2) (by omega)
9
10 -- Subtract: a - b = [l1-h2, h1-l2]
11 def Interval.sub (a b : Interval) : Interval :=
12   match a, b with
13   | .empty, _ | _, .empty => .empty
14   | .range l1 h1 _, .range l2 h2 _ =>
15     .range (l1 - h2) (h1 - l2) (by omega)
16
17 -- Intersect (meet in the information order – more precise)
18 def Interval.intersect (a b : Interval) : Interval :=
19   match a, b with
20   | .empty, _ | _, .empty => .empty
21   | .range l1 h1 _, .range l2 h2 _ =>
22     let l := max l1 l2
23     let h := min h1 h2
24     if h_le : l ≤ h then .range l h h_le else .empty
25
26 end Intervals
```

7.3 A Worked Example: Bound Propagation

Example — $x + y = 10$, $x \in [2, 8]$

What is the tightest interval for y ?

From $x + y = 10$ we get $y = 10 - x$. Since $x \in [2, 8]$, we have $y \in [10 - 8, 10 - 2] = [2, 8]$. The propagator should derive this automatically.

```

1 namespace Intervals
2
3 -- Propagator: sum = x + y (bidirectional: also narrows x and y)
4 def addConstraint
5   (cx cy csum : Propagators.Cell Interval) : Array Propagators.PropStep :=
6   #[
7     -- Forward: sum ← x + y
8     do
9       let x ← cx.read; let y ← cy.read
10      csum.write (Interval.add x y),
11     -- Backward: x ← sum - y
12     do
13       let s ← csum.read; let y ← cy.read
14       cx.write (Interval.sub s y),
15     -- Backward: y ← sum - x
16     do
17       let s ← csum.read; let x ← cx.read
18       cy.write (Interval.sub s x)
19   ]
20
21 def intervalExample : IO Unit := do
22   let cx ← Propagators.Cell.new (α := Interval)
23   let cy ← Propagators.Cell.new (α := Interval)
24   let csum ← Propagators.Cell.new (α := Interval)
25
26   -- x ∈ [2, 8], sum = 10
27   let _ ← cx.write (.range 2 8 (by omega))
28   let _ ← csum.write (.range 10 10 (by omega))
29
30   let steps := addConstraint cx cy csum
31   Propagators.runToFixpoint steps
32
33   let yVal ← cy.read
34   match yVal with
35   | .empty      => IO.println "y: impossible"
36   | .range l h _ => IO.println s!"y ∈ [{l}, {h}]" -- y ∈ [2, 8]
37

```

The join in action. When the backward propagator writes $[2, 8]$ to c_y and c_y already holds $\perp$, the join $\perp \sqcup [2, 8] = [2, 8]$ correctly updates the cell. On the next pass, the forward propagator reads $x = [2, 8]$ and $y = [2, 8]$, computes $\text{add} = [4, 16]$, and writes to c_{sum} . Since $[10, 10] \sqcup [4, 16] = [4, 16]$ would *widen* the constraint (lose information), we could use `intersect` here for a more powerful solver. We leave this as Exercise 7.1.

7.4 Exercises

Exercise 7.1 — Intersection for tighter bounds

Replace `Interval.sup` (widening join) with `Interval.intersect` (narrowing meet) in the cell write operation for the sum constraint. Explain why the information order must be *reversed* when you switch from widening to narrowing. What does this say about the dual lattice?

Exercise 7.2 — Multiplication intervals

Implement `Interval.mul`. Be careful: multiplying negative intervals requires considering all four corner products.

Part III

Capstone Projects

Chapter 8

Capstone I: A Sudoku Solver

8.1 What is Sudoku?

Sudoku is a logic puzzle played on a 9×9 grid. The grid is divided into nine 3×3 *boxes*. Some cells are pre-filled with digits; your task is to fill in the rest.

r1	5	3	1	6	7	8	9	4	2
r2	6	top-left box	2	1	9	5	4	3	8
r3	1	9	8	9	4	6	6	5	7
r4	8				6				3
r5	4			8		3			1
r6	7				2				6
r7		6					2	8	
r8				4	1	9			5
r9					8			7	9
	c1	c2	c3	c4	c5	c6	c7	c8	c9

Figure 8.1: A Sudoku puzzle. Bold digits are givens; light grey digits show a few solved cells. The thick lines divide the grid into nine 3×3 boxes.

The three rules. Every digit 1–9 must appear *exactly once* in each of:

- each **row** (9 rows, one per horizontal strip),
- each **column** (9 columns, one per vertical strip),
- each **box** (9 boxes, each a 3×3 sub-grid).

That is the entire rule set. No arithmetic is involved — the digits could equally well be colours, letters, or symbols. The puzzle is purely about placement constraints.

How to solve by hand. The standard technique is *elimination*: for each empty cell, maintain the set of digits that could still legally go there given the filled cells in the same row, column, and box. If a cell's candidate set shrinks to a single digit, that digit is determined. Filling it in eliminates that digit from the candidate sets of neighbouring cells, which may in turn determine more cells. This cascading process is exactly what a propagator network formalises.

Example — A simple deduction

Suppose row 1 already contains 1, 2, 3, 4, 6, 7, 8, 9 in various cells. The only missing digit is 5, so the one remaining empty cell in row 1 must be 5 — no other placement is consistent with the row constraint.

Example — Elimination across constraints

Suppose a cell sits in row 3, column 5, box 5 (the centre box). Row 3 already contains {1, 3, 4, 6, 7}, column 5 contains {2, 3, 6, 8}, and box 5 contains {1, 2, 5, 6, 9}. The candidates for this cell are all digits *not* ruled out by any of its three units:

$$\{1, \dots, 9\} \setminus \{1, 3, 4, 6, 7\} \setminus \{2, 3, 6, 8\} \setminus \{1, 2, 5, 6, 9\} = \{5\} \setminus \{5\} \cup \dots$$

Working through it: combining the three sets of forbidden digits gives {1, 2, 3, 4, 5, 6, 7, 8, 9} minus what is already placed. The intersection of three elimination sets can be computed with `CandidateSet.inter`, which is exactly the join in the information order.

Why propagator networks are a natural fit. Each constraint (row, column, box) is a propagator that reads the determined cells in its unit and writes the resulting eliminations to undetermined cells. The network runs until quiescence — no propagator can make further eliminations. This is guaranteed to terminate because candidate sets can only shrink, and a finite set can only shrink finitely many times. Whether it *solves* the puzzle completely depends on the puzzle; harder puzzles require search in addition to propagation.

8.2 The Domain Lattice

Sudoku is a constraint satisfaction problem: each of 81 cells must receive a digit 1–9 subject to row, column, and box uniqueness constraints. Propagator networks handle this beautifully. We build a complete solver using nothing but lattice-ordered propagation.

Each Sudoku cell can be modelled as a set of *candidate digits*. Initially every cell can hold any of {1, ..., 9}. Constraints eliminate candidates.

Definition — Candidate Set Lattice

Let $D = \{1, \dots, 9\}$. The *candidate-set lattice* is the powerset $\mathcal{P}(D)$ ordered by *reverse inclusion*:

$$S \leq T \iff T \subseteq S.$$

The bottom $\perp = D$ (no information: any digit possible), the top $\top = \emptyset$ (contradiction: no digit possible). The join is intersection: $S \sqcup T = S \cap T$ (the smallest set of candidates consistent with both sources).

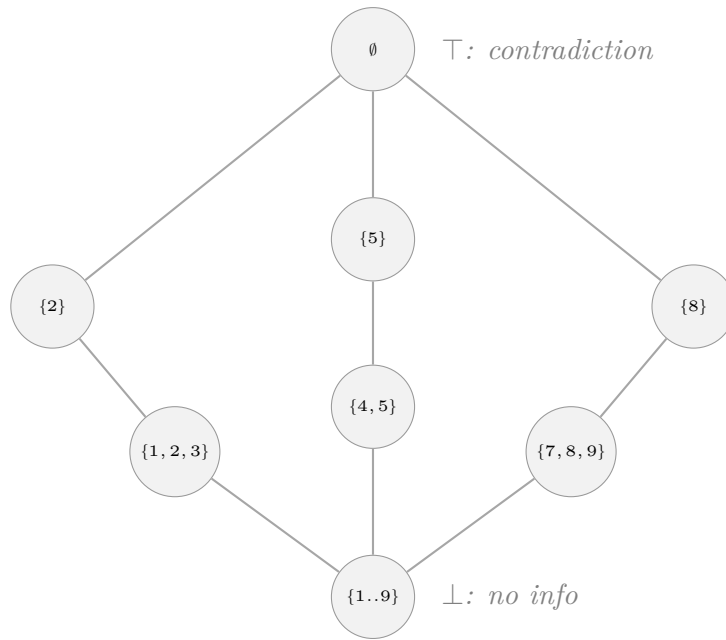


Figure 8.2: A fragment of the candidate-set lattice (reverse inclusion order).

This is precisely the *dual* powerset lattice from Exercise 1.4!

```

1 namespace Sudoku
2
3 -- Digits 1..9, represented as Fin 9 (0..8 internally, displayed as 1..9)
4 abbrev Digit := Fin 9
5
6 -- A candidate set: which digits are still possible for this cell
7 -- We represent it as a Bool array of length 9
8 structure CandidateSet where
9   present : Fin 9 → Bool
10
11 def CandidateSet.full : CandidateSet :=
12   { present := fun _ => true }
13
14 def CandidateSet.empty : CandidateSet :=
15   { present := fun _ => false }
16
17 def CandidateSet.singleton (d : Digit) : CandidateSet :=
18   { present := fun i => i == d }
19
20 def CandidateSet.remove (s : CandidateSet) (d : Digit) : CandidateSet :=
21   { present := fun i => s.present i && !(i == d) }
22
23 def CandidateSet.inter (s t : CandidateSet) : CandidateSet :=
24   { present := fun i => s.present i && t.present i }
25

```

```

26 def CandidateSet.union (s t : CandidateSet) : CandidateSet :=
27   { present := fun i => s.present i || t.present i }
28
29 -- The information order:  $s \leq t$  iff  $t \subseteq s$  ( $t$  is more specific)
30 def CandidateSet.le (s t : CandidateSet) : Prop :=
31    $\forall i : \text{Digit}, t.\text{present } i = \text{true} \rightarrow s.\text{present } i = \text{true}$ 
32
33 instance : OrderBook.PartialOrder CandidateSet where
34   le := CandidateSet.le
35   le_refl := fun s i h => h
36   le_antisymm := fun s t h1 h2 =>
37     show s = t from by
38       cases s; cases t
39       congr 1; funext i
40       cases hi : t.present i
41       · simp [CandidateSet.le] at h1
42       cases hs : s.present i
43       · rfl
44       · exact absurd (h1 i hs) (by simp [hi])
45       · exact h2 i hi
46   le_trans := fun s t u hst htui hi => hst i (htui i hi)
47
48 -- Join = intersection (combines constraints)
49 instance : Propagators.BoundedJoinSemilattice CandidateSet where
50   le := CandidateSet.le
51   le_refl := fun s i h => h
52   le_antisymm := OrderBook.PartialOrder.le_antisymm ( $\alpha := \text{CandidateSet}$ )
53   le_trans := OrderBook.PartialOrder.le_trans ( $\alpha := \text{CandidateSet}$ )
54   bot := CandidateSet.full
55   sup := CandidateSet.inter --  $\sqcup = \cap$  in information order!
56   bot_le := fun s i _ => rfl -- full  $\leq s$  since  $s \subseteq \text{full}$ 
57   le_sup_left := fun s t i h => (Bool.and_eq_true.mp h).1
58   le_sup_right := fun s t i h => (Bool.and_eq_true.mp h).2
59   sup_le := fun s t u hsu htui h =>
60     Bool.and_eq_true.mpr (hsu i h, htui i h)
61
62 instance : DecidableEq CandidateSet :=
63   fun s t =>
64     if h :  $\forall i, s.\text{present } i = t.\text{present } i$ 
65     then isTrue (by cases s; cases t; congr 1; funext i; exact h i)
66     else isFalse (fun heq => h (fun i => by rw [heq]))
67
68 end Sudoku

```

8.3 Representing the Grid

```

1 namespace Sudoku
2
3 -- 81 cells indexed by row and column (both Fin 9)
4 abbrev Grid := Fin 9 → Fin 9 → Propagators.Cell CandidateSet
5
6 def makeGrid : IO Grid := do
7   let mut rows : Array (Array (Propagators.Cell CandidateSet)) := #[]
8   for _ in [:9] do
9     let mut row : Array (Propagators.Cell CandidateSet) := #[]
10    for _ in [:9] do
11      let c ← Propagators.Cell.new (α := CandidateSet)
12      row := row.push c
13    rows := rows.push row
14  -- Convert to function (safe because sizes match)
15  return fun r c => rows[r.val]![c.val]!
16
17 -- Seed a given digit into a cell
18 def seedCell (grid : Grid) (r c : Fin 9) (d : Digit) : IO Unit := do
19   let _ ← (grid r c).write (CandidateSet.singleton d)
20
21 end Sudoku

```

8.4 Constraint Propagators

Sudoku has three families of constraints: row, column, and 3×3 box uniqueness. Each is expressed as a propagator that, given a cell known to hold digit d , removes d from all peers.

```

1 namespace Sudoku
2
3 -- If cell (r,c) is determined to be d, remove d from all other cells in row r
4 def rowPropagator (grid : Grid) (r c : Fin 9) : Propagators.PropStep := do
5   let cands ← (grid r c).read
6   -- Check if this cell is a singleton
7   let det : Option Digit := Id.run do
8     let mut found : Option Digit := none
9     let mut count := 0
10    for i in [:9] do
11      let i' : Digit := ⟨i, by omega⟩
12      if cands.present i' then
13        found := some i'; count := count + 1

```

```

14     if count = 1 then found else none
15 match det with
16 | none => return false
17 | some d =>
18     let mut changed := false
19     for j in [:9] do
20         let j' : Fin 9 := ⟨j, by omega⟩
21         if j' ≠ c then
22             let removed := { present := fun i => (grid r j').read.present i && !(i
23                 ↪ == d) }
24             -- We re-read and merge
25             let old ← (grid r j').read
26             let upd := CandidateSet.inter old { present := fun i => !(i == d) }
27             let c ← (grid r j').write upd
28             if c then changed := true
29     return changed
30 -- Analogous: column propagator
31 def colPropagator (grid : Grid) (r c : Fin 9) : Propagators.PropStep := do
32     let cands ← (grid r c).read
33     let det : Option Digit := Id.run do
34         let mut found : Option Digit := none
35         let mut count := 0
36         for i in [:9] do
37             let i' : Digit := ⟨i, by omega⟩
38             if cands.present i' then found := some i'; count := count + 1
39         if count = 1 then found else none
40     match det with
41 | none => return false
42 | some d =>
43     let mut changed := false
44     for i in [:9] do
45         let i' : Fin 9 := ⟨i, by omega⟩
46         if i' ≠ r then
47             let old ← (grid i' c).read
48             let upd := CandidateSet.inter old { present := fun k => !(k == d) }
49             let ch ← (grid i' c).write upd
50             if ch then changed := true
51     return changed
52
53 -- Box propagator (which 3×3 box contains (r,c)?)
54 def boxPropagator (grid : Grid) (r c : Fin 9) : Propagators.PropStep := do
55     let cands ← (grid r c).read
56     let det : Option Digit := Id.run do
57         let mut found : Option Digit := none; let mut count := 0
58         for i in [:9] do

```

```

59     let i' : Digit := (i, by omega)
60     if candS.present i' then found := some i'; count := count + 1
61     if count = 1 then found else none
62 match det with
63 | none => return false
64 | some d =>
65     let boxR := r.val / 3 * 3
66     let boxC := c.val / 3 * 3
67     let mut changed := false
68     for dr in [:3] do
69         for dc in [:3] do
70             let r' : Fin 9 := (boxR + dr, by omega)
71             let c' : Fin 9 := (boxC + dc, by omega)
72             if !(r' = r && c' = c) then
73                 let old ← (grid r' c').read
74                 let upd := CandidateSet.inter old { present := fun k => !(k == d) }
75                 let ch ← (grid r' c').write upd
76                 if ch then changed := true
77     return changed
78
79 end Sudoku

```

8.5 Assembling and Running the Solver

```

1 namespace Sudoku
2
3 def buildNetwork (grid : Grid) : Array Propagators.PropStep := Id.run do
4     let mut steps : Array Propagators.PropStep := #[]
5     for r in [:9] do
6         for c in [:9] do
7             let r' : Fin 9 := (r, by omega)
8             let c' : Fin 9 := (c, by omega)
9             steps := steps.push (rowPropagator grid r' c')
10            steps := steps.push (colPropagator grid r' c')
11            steps := steps.push (boxPropagator grid r' c')
12    return steps
13
14 -- Print the grid (showing the unique digit or '?' for unsolved cells)
15 def printGrid (grid : Grid) : IO Unit := do
16     for r in [:9] do
17         let mut row := ""
18         for c in [:9] do
19             let r' : Fin 9 := (r, by omega)

```

```
20     let c' : Fin 9 := (c, by omega)
21     let cands ← (grid r' c').read
22     let mut digit := '?'
23     let mut count := 0
24     for i in [:9] do
25         let i' : Digit := (i, by omega)
26         if cands.present i' then digit := Char.ofNat (i + 49); count := count
27             ↪ + 1
28         row := row ++ (if count = 1 then s!"{digit}" else "?")
29         if c % 3 == 2 && c < 8 then row := row ++ "|"
30     IO.println row
31     if r % 3 == 2 && r < 8 then IO.println "-----"
32
33 -- Example: a simple Sudoku puzzle
34 -- (0 = unknown, digits are given clues)
35 def examplePuzzle : Array (Array Nat) := #[
36     #[5, 3, 0, 0, 7, 0, 0, 0, 0],
37     #[6, 0, 0, 1, 9, 5, 0, 0, 0],
38     #[0, 9, 8, 0, 0, 0, 0, 6, 0],
39     #[8, 0, 0, 0, 6, 0, 0, 0, 3],
40     #[4, 0, 0, 8, 0, 3, 0, 0, 1],
41     #[7, 0, 0, 0, 2, 0, 0, 0, 6],
42     #[0, 6, 0, 0, 0, 0, 2, 8, 0],
43     #[0, 0, 0, 4, 1, 9, 0, 0, 5],
44     #[0, 0, 0, 0, 8, 0, 0, 7, 9]
45 ]
46
47 def solveSudoku : IO Unit := do
48     let grid ← makeGrid
49     -- Seed known digits
50     for r in [:9] do
51         for c in [:9] do
52             let d := examplePuzzle[r][c]!
53             if d ≠ 0 then
54                 let r' : Fin 9 := (r, by omega)
55                 let c' : Fin 9 := (c, by omega)
56                 seedCell grid r' c' (d - 1, by omega)
57         -- Build and run the propagator network
58     let steps := buildNetwork grid
59     Propagators.runToFixpoint steps
60     -- Print the result
61     IO.println "Solution:"
62     printGrid grid
63 end Sudoku
```

8.6 Why This Works: The Lattice Perspective

- Each cell starts at $\perp = \{1, \dots, 9\}$: no information.
- Every propagator only writes using `CandidateSet.inter` (the join in the information order), which can only *reduce* candidate sets — i.e., move *up* in the information order.
- The network terminates because the candidate-set lattice is finite and the cells only move upward.
- Reaching $\top = \emptyset$ in any cell signals a contradiction.
- A solved cell is one that has reached a singleton $\{d\}$.

The elegant insight: *solving Sudoku is computing the least fixed point of the combined propagation function*, where the partial order is the information order on candidate sets. Knaster–Tarski guarantees it exists; our scheduler computes it.

Exercise 8.1 — Naked pair

A *naked pair* occurs when two cells in the same unit both have exactly the same two candidates $\{a, b\}$. Then a and b can be removed from all other cells in that unit. Implement this as an additional propagator.

Exercise 8.2 — Contradiction detection

Add a `contradictionProp` that checks every cell; if any cell reaches $\top = \emptyset$, print a diagnostic and halt. Why is this detection sound (i.e., why can't a cell reach $\emptyset$ unless the puzzle is truly unsatisfiable)?

Chapter 9

Capstone II: Type Inference via Propagation

Our second capstone takes us into programming language theory. Type inference — the process of automatically deducing types of expressions — is naturally formulated as constraint propagation on a type lattice.

9.1 A Simple Expression Language

We work with a fragment of the lambda calculus extended with integers and booleans.

```
1 namespace TypeInfer
2
3 -- Expressions
4 inductive Expr where
5   | intLit (n : Int)      : Expr
6   | boolLit (b : Bool)   : Expr
7   | var     (x : String)  : Expr
8   | add     (e1 e2 : Expr) : Expr
9   | ifExpr  (cond t f : Expr) : Expr
10  | lam     (x : String) (body : Expr) : Expr
11  | app     (fn arg : Expr) : Expr
12  | letExpr (x : String) (e body : Expr) : Expr
13
14 end TypeInfer
```

9.2 The Type Lattice

Rather than types being a simple set, we arrange them in a lattice that captures the degree to which a type is *known*.

Definition — Flat Type Lattice

Define the type information domain:

- $\perp$ (Unknown): no type information yet.
- TInt , TBool : concrete base types.
- $\text{TFun } s \ t$: function type, parameterised by the type information for the domain and codomain (which are themselves in the lattice).
- $\top$ (TypeError): contradiction (incompatible constraints).

The order is: $\perp \leq$ everything, everything $\leq \top$, and base types are only $\leq$ themselves (and $\perp/\top$). Function types:

$\text{TFun } s_1 \ t_1 \leq \text{TFun } s_2 \ t_2$ iff $s_1 \leq s_2 \wedge t_1 \leq t_2$.

```

1 namespace TypeInfer
2
3 inductive Ty where
4   | unknown      : Ty    --  $\perp$ 
5   | int          : Ty
6   | bool         : Ty
7   | fun_ (dom cod : Ty) : Ty    --  $\text{dom} \rightarrow \text{cod}$ 
8   | typeError    : Ty    --  $\top$ 
9
10 -- Unification of two types: compute their join
11 def Ty.unify : Ty → Ty → Ty
12   | .unknown, t      => t
13   | t, .unknown      => t
14   | .typeError, _    => .typeError
15   | _, .typeError    => .typeError
16   | .int, .int       => .int
17   | .bool, .bool     => .bool
18   | .fun_ d1 c1, .fun_ d2 c2 => .fun_ (Ty.unify d1 d2) (Ty.unify c1 c2)
19   | _, _            => .typeError -- incompatible types
20
21 -- The information order
22 def Ty.le : Ty → Ty → Prop
23   | .unknown, _      => True
24   | _, .typeError    => True
25   | .int, .int       => True
26   | .bool, .bool     => True
27   | .fun_ d1 c1, .fun_ d2 c2 => Ty.le d1 d2 ∧ Ty.le c1 c2
28   | _, _            => False
29
30 instance : OrderBook.PartialOrder Ty where
31   le      := Ty.le
32   le_refl := by intro t; induction t <;> simp [Ty.le, *]
33   le_antisymm := by
34     intro t1 t2 h12 h21

```

```

35   induction t1 <=> cases t2 <=> simp [Ty.le] at * <=>
36     try (first | rfl | exact absurd h12 id | exact absurd h21 id)
37     case fun_.fun_ d1 c1 ih_d ih_c d2 c2 =>
38       have := ih_d h12.1 h21.1; have := ih_c h12.2 h21.2; simp_all
39   le_trans      := by
40     intro t1 t2 t3 h12 h23
41     induction t1 <=> cases t2 <=> cases t3 <=>
42       simp [Ty.le] at * <=> try tauto
43     case fun_.fun_.fun_ => exact (by tauto, by tauto)
44
45 instance : Propagators.BoundedJoinSemilattice Ty where
46   le      := Ty.le
47   le_refl := OrderBook.PartialOrder.le_refl
48   le_antisymm := OrderBook.PartialOrder.le_antisymm
49   le_trans := OrderBook.PartialOrder.le_trans
50   bot      := .unknown
51   sup      := Ty.unify
52   bot_le   := fun _ => trivial
53   le_sup_left := by
54     intro a b; induction a <=> cases b <=>
55       simp [Ty.le, Ty.unify, *] <=> try exact (by assumption, by assumption)
56   le_sup_right := by
57     intro a b; induction b <=> cases a <=>
58       simp [Ty.le, Ty.unify, *] <=> try exact (by assumption, by assumption)
59   sup_le      := by
60     intro a b c hac hbc
61     induction a <=> cases b <=> cases c <=>
62       simp [Ty.le, Ty.unify] at * <=> tauto
63
64 deriving instance DecidableEq for Ty
65
66 end TypeInfer

```

9.3 Type Constraint Generation

We generate constraints by walking the expression tree. Each sub-expression gets a fresh *type cell*. Constraints are propagators that merge type information into these cells.

```

1 namespace TypeInfer
2
3 -- A typing context maps variable names to type cells
4 abbrev Context := List (String × Propagators.Cell Ty)
5

```

```
6 def Context.lookup (ctx : Context) (x : String) :
7   Option (Propagators.Cell Ty) :=
8   ctx.find? (fun p => p.1 == x) |>.map (·.2)
9
10 -- Generate type cells and propagators for an expression
11 -- Returns the type cell for this expression.
12 partial def inferExpr
13   (ctx : Context)
14   (e : Expr)
15   (steps : IO.Ref (Array Propagators.PropStep)) :
16   IO (Propagators.Cell Ty) := do
17   match e with
18   | .intLit _ =>
19     let c ← Propagators.Cell.new (α := Ty)
20     let _ ← c.write .int
21     return c
22
23   | .boolLit _ =>
24     let c ← Propagators.Cell.new (α := Ty)
25     let _ ← c.write .bool
26     return c
27
28   | .var x =>
29     match ctx.lookup x with
30     | some c => return c
31     | none   =>
32       let c ← Propagators.Cell.new (α := Ty)
33       let _ ← c.write .typeError -- unbound variable
34       return c
35
36   | .add e1 e2 =>
37     let c1 ← inferExpr ctx e1 steps
38     let c2 ← inferExpr ctx e2 steps
39     let cOut ← Propagators.Cell.new (α := Ty)
40     -- Propagator: both inputs and output must be Int
41     steps.modify (· .push do
42       let _ ← c1.write .int
43       let _ ← c2.write .int
44       cOut.write .int)
45     return cOut
46
47   | .ifExpr cond t f =>
48     let ccond ← inferExpr ctx cond steps
49     let ct    ← inferExpr ctx t     steps
50     let cf    ← inferExpr ctx f     steps
51     let cOut  ← Propagators.Cell.new (α := Ty)
```

```

52  -- Propagator: cond must be Bool; t and f must agree; out = t
53  steps.modify (·.push do
54    let _ ← ccond.write .bool
55    let tv ← ct.read; let fv ← cf.read
56    let _ ← ct.write fv; let _ ← cf.write tv
57    let tv' ← ct.read
58    cOut.write tv')
59  return cOut
60
61  | .lam x body =>
62    let cDom ← Propagators.Cell.new (α := Ty)
63    let ctx' := (x, cDom) :: ctx
64    let cBody ← inferExpr ctx' body steps
65    let cOut ← Propagators.Cell.new (α := Ty)
66    -- Propagator: out = dom → body
67    steps.modify (·.push do
68      let d ← cDom.read; let b ← cBody.read
69      cOut.write (.fun_ d b))
70    return cOut
71
72  | .app fn arg =>
73    let cFn ← inferExpr ctx fn steps
74    let cArg ← inferExpr ctx arg steps
75    let cOut ← Propagators.Cell.new (α := Ty)
76    -- Propagator: fn must be (arg → out)
77    steps.modify (·.push do
78      let fv ← cFn.read; let av ← cArg.read
79      let ov ← cOut.read
80      match fv with
81      | .fun_ d r =>
82        let _ ← cArg.write d -- arg type must match domain
83        let _ ← cOut.write r -- result type from codomain
84        let _ ← cFn.write (.fun_ av ov) -- refine fn type
85        return false -- (actual change tracked by writes)
86      | .unknown =>
87        let _ ← cFn.write (.fun_ av ov)
88        return false
89      | _ =>
90        let _ ← cFn.write .typeError; return false)
91    return cOut
92
93  | .letExpr x e body =>
94    let ce ← inferExpr ctx e steps
95    let ctx' := (x, ce) :: ctx
96    inferExpr ctx' body steps
97

```

```
98 end TypeInfer
```

9.4 Running the Type Inferencer

```
1 namespace TypeInfer
2
3 def infer (e : Expr) : IO Ty := do
4   let stepsRef ← IO.mkRef (Array.empty : Array Propagators.PropStep)
5   let cResult  ← inferExpr [] e stepsRef
6   let steps    ← stepsRef.get
7   Propagators.runToFixpoint steps
8   cResult.read
9
10  -- Helper to show types
11  def Ty.toString : Ty → String
12    | .unknown      => "?"
13    | .int          => "Int"
14    | .bool         => "Bool"
15    | .fun_ d r     => s! "{Ty.toString d} → {Ty.toString r}"
16    | .typeError    => "TypeError"
17
18  -- Examples
19  def ex1 : Expr := .add (.intLit 1) (.intLit 2)
20  -- Expected: Int
21
22  def ex2 : Expr := .lam "x" (.add (.var "x") (.intLit 1))
23  -- Expected: (Int → Int)
24
25  def ex3 : Expr := .ifExpr (.boolLit true) (.intLit 1) (.intLit 2)
26  -- Expected: Int
27
28  def ex4 : Expr := .add (.boolLit true) (.intLit 1)
29  -- Expected: TypeError (Bool ≠ Int)
30
31  def runExamples : IO Unit := do
32    for (name, e) in [("1 + 2", ex1), ("λx. x+1", ex2),
33                     ("if true 1 2", ex3), ("true + 1", ex4)] do
34      let ty ← infer e
35      IO.println s! "{name} : {Ty.toString ty}"
36
37  end TypeInfer
```

Expected output:

```
1 + 2 : Int
λx. x+1 : (Int → Int)
if true 1 2 : Int
true + 1 : TypeError
```

9.5 What the Lattice Buys Us Here

- **Bidirectional propagation is free.** When we call `cArg.write d` inside the `app` propagator, we are propagating type information *backwards*: from the function type to the argument. The lattice join ensures this never loses previously-inferred information.
- **Inconsistency is explicit.** `TypeError` is the $\top$ element. When two incompatible type constraints are merged (`Ty.unify .int .bool = .TypeError`), the cell value jumps to $\top$ and stays there. No exception is thrown; the type error propagates naturally through the lattice.
- **Incrementality is free.** If we learn a new fact about x 's type (say, x appears in another expression), we just write to x 's type cell and re-run the network. The lattice join handles merging old and new constraints correctly.
- **Ordering captures “how much we know”.** $\perp = \text{Unknown} < \text{Int} < \text{TypeError} = \top$. The height of a cell's value in the lattice is a measure of how constrained it is. A fully-typed program has all cells at concrete types (neither $\perp$ nor $\top$).

9.6 Exercises

Exercise 9.1 — Let-polymorphism

Lean's type system supports polymorphism. Extend `Ty` with a `TVar (name : String)` constructor and modify `Ty.unify` to do proper unification (substituting type variables). How does this affect the lattice structure?

Exercise 9.2 — Recursive types

Add a `letRec` construct to `Expr` and extend the inferencer to handle it. Note: the type cell for the recursive binding must be initialised to `Unknown` and refined by propagation, enabling inference of e.g. `let rec f x = if x = 0 then 1 else x * f (x-1) .`

Exercise 9.3 — Error messages

When a cell reaches `TypeError`, record *why* (which two types conflicted). Augment the `Ty` type so that `TypeError` carries a `String` explanation, and thread error messages through `Ty.unify`.

Appendix A

Lean 4 Quick Reference

A.1 Basic Types and Syntax

```
1  -- Type universes
2  #check Nat      -- Nat : Type
3  #check Type     -- Type : Type 1
4  #check Prop     -- Prop : Sort 0
5
6  -- Function types
7  def f : Nat → Nat → Nat := fun a b => a + b
8  def g (a b : Nat) : Nat := a + b    -- same, different syntax
9
10 -- Dependent types
11 def myList (α : Type) (n : Nat) : Type := ...
12
13 -- Let bindings
14 def h : Nat :=
15   let x := 5
16   let y := x + 1
17   x * y
```

A.2 Typeclasses

```
1  -- Declaring a typeclass
2  class MyClass (α : Type) where
3    someMethod : α → Nat
4    someProperty : ∀ (a : α), someMethod a ≥ 0
5
6  -- Providing an instance
7  instance : MyClass Nat where
8    someMethod := id
9    someProperty := Nat.zero_le
10
```

```
11 -- Using an instance
12 def useMyClass [MyClass  $\alpha$ ] (a :  $\alpha$ ) : Nat :=
13   MyClass.someMethod a
```

A.3 Tactics Reference

Tactic	Usage
rfl	Prove $a = a$ by reflexivity
exact h	Close goal with proof term h
apply f	Apply function/lemma f
intro h	Introduce a hypothesis
cases h	Case split on h
induction n	Induction on n
simp	Simplify using simp lemmas
simp_all	Simplify using all hypotheses
omega	Linear arithmetic over $\mathbb{Z}$
decide	Decide propositions by computation
constructor	Introduce an And/Iff/...
have h := ...	Introduce a local lemma
tauto	Tautology checker
funext	Function extensionality
congr	Congruence

Table A.1: Commonly used Lean 4 tactics in this book.

A.4 Unicode Input

Symbol	Input	Meaning
$\leq$	<code>\le</code>	Less-than-or-equal
$\geq$	<code>\ge</code>	Greater-than-or-equal
$\rightarrow$	<code>\to</code>	Function/implication arrow
$\leftarrow$	<code>\l</code>	Reverse arrow
$\leftrightarrow$	<code>\iff</code>	If and only if
$\forall$	<code>\fa</code>	For all
$\exists$	<code>\ex</code>	There exists
$\sqcap$	<code>\sqcap</code>	Meet (inf)
$\sqcup$	<code>\sqcup</code>	Join (sup)
$\perp$	<code>\bot</code>	Bottom
$\top$	<code>\top</code>	Top
$\in$	<code>\in</code>	Set membership
$\notin$	<code>\notin</code>	Non-membership
α	<code>\a</code>	Greek alpha
β	<code>\b</code>	Greek beta
$\mathbb{N}$	<code>\N</code>	Natural numbers
$\wedge$	<code>\and</code>	Logical and
$\vee$	<code>\or</code>	Logical or
$\neg$	<code>\n</code>	Logical not

Table A.2: Lean 4 Unicode inputs used in this book.

Selected Solutions

This appendix contains solutions to exercises with a single definitive Lean 4 answer. Open-ended exercises are left to the reader.

Chapter 1

Exercise 1.1 (Reflexivity and symmetry)

```
1 theorem symm_antisymm_eq {α : Type} (r : Rel α)
2   (hs : Symmetric r) (ha : Antisymm r) :
3   ∀ a b : α, r a b → a = b := by
4   intro a b h
5   exact ha a b h (hs a b h)
```

Exercise 1.3 (Product order)

```
1 instance [OrderBook.PartialOrder α] [OrderBook.PartialOrder β] :
2   OrderBook.PartialOrder (α × β) where
3   le p q := PartialOrder.le p.1 q.1 ∧ PartialOrder.le p.2 q.2
4   le_refl := fun ⟨a, b⟩ =>
5     ⟨PartialOrder.le_refl a, PartialOrder.le_refl b⟩
6   le_antisymm := fun ⟨a1,b1⟩ ⟨a2,b2⟩ ⟨h1,h2⟩ ⟨h3,h4⟩ => by
7     have ha := PartialOrder.le_antisymm _ _ h1 h3
8     have hb := PartialOrder.le_antisymm _ _ h2 h4
9     simp_all
10  le_trans := fun ⟨a1,b1⟩ ⟨a2,b2⟩ ⟨a3,b3⟩ ⟨h1,h2⟩ ⟨h3,h4⟩ =>
11    ⟨PartialOrder.le_trans _ _ _ h1 h3,
12      PartialOrder.le_trans _ _ _ h2 h4⟩
```

Chapter 2

Exercise 2.1 (Monotone composition)

```
1 theorem antitone_comp_antitone {α β γ : Type}
2   [LE α] [LE β] [LE γ]
```

```
3   [PartialOrder  $\alpha$ ] [PartialOrder  $\beta$ ] [PartialOrder  $\gamma$ ]
4   {f :  $\alpha \rightarrow \beta$ } {g :  $\beta \rightarrow \gamma$ }
5   (hf :  $\forall a b : \alpha, a \leq b \rightarrow f b \leq f a$ )
6   (hg :  $\forall a b : \beta, a \leq b \rightarrow g b \leq g a$ ) :
7   Monotone (g  $\circ$  f) where
8   map_le := fun a b h => hg (f b) (f a) (hf a b h)
```

Exercise 2.2 (Fixed points — ascending chain)

```
1 def iter { $\alpha$  : Type} (f :  $\alpha \rightarrow \alpha$ ) : Nat  $\rightarrow$   $\alpha \rightarrow \alpha$ 
2   | 0,      x => x
3   | n + 1, x => f (iter f n x)
4
5 theorem iter_chain { $\alpha$  : Type} [LE  $\alpha$ ] [PartialOrder  $\alpha$ ]
6   {f :  $\alpha \rightarrow \alpha$ } (hf : Monotone f)
7   {a :  $\alpha$ } (ha :  $a \leq f a$ ) :
8    $\forall n : \text{Nat}, \text{iter } f \ n \ a \leq \text{iter } f \ (n + 1) \ a := \text{by}$ 
9   intro n
10  induction n with
11  | zero => exact ha
12  | succ n ih => exact hf.map_le _ _ ih
```

Exercise 2.3 (Identity is monotone)

```
1 theorem Monotone.id { $\alpha$  : Type} [LE  $\alpha$ ] [PartialOrder  $\alpha$ ] :
2   Monotone (id :  $\alpha \rightarrow \alpha$ ) where
3   map_le := fun _ _ h => h
```

Chapter 3

Exercise 3.1 (Interval partial order)

```
1 structure Interval where
2   l      : Int
3   h      : Int
4   l_le_h : l  $\leq$  h
5
6 instance : LE Interval where
7   le := fun i j => j.l  $\leq$  i.l  $\wedge$  i.h  $\leq$  j.h
8
9 instance : PartialOrder Interval where
```

```

10 le_refl := fun a => ⟨Int.le_refl a.l, Int.le_refl a.h⟩
11 le_antisymm {a b} := fun hyp1 hyp2 => by
12   cases a; cases b
13   rename_i l1 h1 l1_le_h1 l2 h2 l2_le_h2
14   congr 1
15   · exact Int.le_antisymm hyp2.left hyp1.left
16   · exact Int.le_antisymm hyp1.right hyp2.right
17   -- l_le_h closed by proof irrelevance
18 le_trans {a b c} := fun hyp1 hyp2 => by
19   rcases a with ⟨a_l, a_h, a_le⟩
20   rcases b with ⟨b_l, b_h, b_le⟩
21   rcases c with ⟨c_l, c_h, c_le⟩
22   have hl_ba : b_l ≤ a_l := hyp1.left
23   have hl_cb : c_l ≤ b_l := hyp2.left
24   have hh_ab : a_h ≤ b_h := hyp1.right
25   have hh_bc : b_h ≤ c_h := hyp2.right
26   exact ⟨Int.le_trans hl_cb hl_ba, Int.le_trans hh_ab hh_bc⟩

```

Exercise 3.3 (Characterising the order)

```

1 theorem le_iff_sup_eq {α : Type} [Lattice α] {a b : α} :
2   a ≤ b ↔ a ⊔ b = b := by
3   apply Iff.intro
4   · intro h
5     apply PartialOrder.le_antisymm
6     · apply Lattice.le_sup
7       · exact h
8       · exact PartialOrder.le_refl b
9     · exact Lattice.sup_le_right a b
10  · intro h
11    rw [← h]
12    exact Lattice.sup_le_left a b

```

Exercise 3.4 (Product lattice)

```

1 instance {α β : Type} [Lattice α] [Lattice β] : Lattice (α × β) where
2   le := fun a b => (a.fst ≤ b.fst) ∧ (a.snd ≤ b.snd)
3   le_refl := fun a => by
4     apply And.intro
5     · apply PartialOrder.le_refl
6     · apply PartialOrder.le_refl
7   le_antisymm {a b} := fun hyp1 hyp2 => by
8     rcases a with ⟨l1, r1⟩; rcases b with ⟨l2, r2⟩; congr 1
9     · exact PartialOrder.le_antisymm hyp1.left hyp2.left

```

```

10   · exact PartialOrder.le_antisymm hyp1.right hyp2.right
11 le_trans {a b c} := fun hyp1 hyp2 => by
12   apply And.intro
13   · exact PartialOrder.le_trans hyp1.left hyp2.left
14   · exact PartialOrder.le_trans hyp1.right hyp2.right
15 inf       := fun a b =>
16   (Lattice.inf a.fst b.fst, Lattice.inf a.snd b.snd)
17 sup       := fun a b =>
18   (Lattice.sup a.fst b.fst, Lattice.sup a.snd b.snd)
19 inf_le_left := fun a b => (Lattice.inf_le_left a.fst b.fst,
20   Lattice.inf_le_left a.snd b.snd)
21 inf_le_right := fun a b => (Lattice.inf_le_right a.fst b.fst,
22   Lattice.inf_le_right a.snd b.snd)
23 le_inf     := fun a b c hyp1 hyp2 =>
24   (Lattice.le_inf a.fst b.fst c.fst hyp1.left hyp2.left,
25   Lattice.le_inf a.snd b.snd c.snd hyp1.right hyp2.right)
26 sup_le_left := fun a b => (Lattice.sup_le_left a.fst b.fst,
27   Lattice.sup_le_left a.snd b.snd)
28 sup_le_right := fun a b => (Lattice.sup_le_right a.fst b.fst,
29   Lattice.sup_le_right a.snd b.snd)
30 le_sup     := fun a b c hyp1 hyp2 =>
31   (Lattice.le_sup a.fst b.fst c.fst hyp1.left hyp2.left,
32   Lattice.le_sup a.snd b.snd c.snd hyp1.right hyp2.right)

```

Chapter 6

Exercise 6.1 (Three-cell multiplication constraint)

The pattern mirrors the addition network: one forward propagator and two backward propagators, one for each variable that can be derived from the other two. Division by zero writes `.conflict`.

```

1 def mulProp (cx cy cz : Cell FlatNat) : PropStep := do
2   let x ← cx.read
3   let y ← cy.read
4   match x, y with
5   | .known a, .known b => cz.write (.known (a * b))
6   | _, _              => return false
7
8 -- z / x → y (or conflict if x | / z)
9 def divProp (cz cx cy : Cell FlatNat) : PropStep := do
10  let z ← cz.read
11  let x ← cx.read
12  match z, x with

```



```

13     | .known z, .known x =>
14       if x | z then cy.write (.known (z / x))
15       else cy.write (.conflict)
16     | _, _ => return false
17
18 def mulNetwork : IO Unit := do
19   let cx ← Cell.new (α := FlatNat)
20   let cy ← Cell.new (α := FlatNat)
21   let cz ← Cell.new (α := FlatNat)
22
23   let _ ← cx.write (.known 3)
24   let _ ← cz.write (.known 12)
25
26   let net : Network := Network.mk #[
27     mulProp cx cy cz,
28     divProp cz cx cy,    -- z / x → y
29     divProp cz cy cx    -- z / y → x
30   ]
31   net.run
32
33   let yVal ← cy.read
34   match yVal with
35   | .known n => IO.println s!"y = {n}"    -- y = 4
36   | .unknown => IO.println "y is unknown"
37   | .conflict => IO.println "contradiction"
38
39 #eval mulNetwork    -- y = 4

```

The three propagators cover all three “solve for one variable” cases. Remove `divProp cz cx cy` and `y` stays unknown; remove `mulProp` and you lose the ability to derive `cz` from both inputs.

Exercise 6.2 (Conflict detection)

Two independent writes of different values to the same cell produce `.conflict` via the join in `Cell.write`: `known 3 ∪ known 5 = conflict`. No propagators are needed — the conflict arises purely from the two seeds.

```

1 def conflictExample : IO Unit := do
2   let cx ← Cell.new (α := FlatNat)
3
4   -- Two independent sources disagree about x
5   let _ ← cx.write (.known 3)
6   let _ ← cx.write (.known 5)
7
8   let xVal ← cx.read

```

```
9   match xVal with
10   | .conflict => IO.println "contradiction (expected)"
11   | .known n  => IO.println s!"x = {n}"
12   | .unknown  => IO.println "x unknown"
13
14 #eval conflictExample    -- contradiction (expected)
```

The key is in `Cell.write`: it merges the new value with the old via `le := ( · · )` -- *Bool has a built-in* so the second write computes `known 3  $\sqcup$  known 5 = conflict` and stores that. From this point any further `write` leaves `conflict` unchanged (since `conflict  $\sqcup$  x = conflict` for all `x`), and any propagator reading `cx` will receive `conflict` and propagate it downstream.

To demonstrate downstream propagation, add a dependent cell:

```
1  def conflictPropagates : IO Unit := do
2    let cx  ← Cell.new (α := FlatNat)
3    let cy  ← Cell.new (α := FlatNat)
4    let csum ← Cell.new (α := FlatNat)
5
6    let _ ← cx.write (.known 3)
7    let _ ← cx.write (.known 5)    -- cx is now conflict
8    let _ ← cy.write (.known 7)
9
10   let net : Network := Network.mk #[addProp cx cy csum]
11   net.run
12
13   let s ← csum.read
14   match s with
15   | .conflict => IO.println "sum is conflict (expected)"
16   | .known n  => IO.println s!"sum = {n}"
17   | .unknown  => IO.println "sum unknown"
18
19 #eval conflictPropagates    -- sum is conflict (expected)
```

Further Reading

- **Davey & Priestley**, *Introduction to Lattices and Order*, 2nd ed., Cambridge University Press, 2002. The standard introductory text on lattice theory, accessible and rigorous.
- **Radul & Sussman**, *The Art of the Propagator*, MIT CSAIL Technical Report, 2009. The foundational paper for the propagator model used in this book.
- **Tate, Stepp, Lerner & Chambers**, *Equality Saturation: a New Approach to Optimization*, POPL 2009. An application of lattice-based fixed-point algorithms to compiler optimization.
- **Nielson, Nielson & Hankin**, *Principles of Program Analysis*, Springer, 1999. Covers dataflow analysis (a lattice-based framework) and abstract interpretation.
- **The Lean 4 Documentation**, <https://lean-lang.org>. Official reference for Lean 4 syntax, tactics, and the core library.
- **Mathlib4**, https://leanprover-community.github.io/mathlib4_docs/. The community mathematics library for Lean 4. Once you have finished this book, exploring `Mathlib.Order` will show you how the constructions here scale to a large, verified library.